DESIGN AND DEVELOPMENT OF AN AUTONOMOUS BASKETBALL PLAYING ROBOT USING AI, ROBOTICS, AND CONTROL SYSTEMS (SMART-BASKETBOT)

Design Experience [I / II / III] Project Report

submitted in partial fulfilment of the requirements of

Bachelor of Technology in Artificial Intelligence

in

Department of Computer Engineering

Vertical: Vertically Integrated Projects (VIP) / Industry Research

by

Neil George (Roll No. [Roll No.] / SAP ID [SAP ID])

Under the supervision of

[Name of Faculty Mentor]

[Designation, Department, MPSTME]



MPSTME, SVKM's NMIMS (Deemed-to-be University), Mumbai 2025–2026

SVKM's NMIMS University

(Deemed-to-be University)



MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (MPSTME)

Vile Parle (W), Mumbai-56

[Academic Year]

CERTIFICATE

This is to certify that the Design Experience III project entitled "DESIGN AND DEVELOPMENT OF AN AUTONOMOUS BASKETBALL PLAYING ROBOT USING AI, ROBOTICS, AND CONTROL SYSTEMS (SMART-BASKETBOT)" has been carried out by Neil George (Roll No. [Roll No.] / SAP ID [SAP ID]) under my guidance and supervision, and has been submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence in Department of Computer Engineering of Mukesh Patel School of Technology Management & Engineering, SVKM's NMIMS (Deemed-to-be University), Mumbai during the academic year 2025–2026.

Project mentor (name and signature) (Internal Guide)	Vertical SPOC (name and signature)	HoD (name and signature)

Date: [Date: April 2026]

Place: Mumbai

ACKNOWLEDGEMENT

I express my deepest gratitude to my faculty mentor, [Name of Faculty Mentor], [Designation, Department, MPSTME], for invaluable guidance, constructive critiques, and constant encouragement throughout the design and realization of the SMART-BASKETBOT project. I also extend sincere appreciation to the Vertical SPOC,

[Name of Vertical SPOC], and the Head of Department, [Name of Head of Department], for providing the institutional cyber-physical laboratory infrastructure and computational resources.

I am grateful to SVKM's NMIMS Mukesh Patel School of Technology Management & Engineering (MPSTME) for fostering an innovative environment under the Design Experience curriculum. Finally, I thank my peers for their intellectual discussions and constructive feedback during sprint demonstrations.

Name	Roll No.	SAP ID
Neil George	[Roll No.]	[SAP ID]

DECLARATION

I declare that this report entitled "DESIGN AND DEVELOPMENT OF AN AUTONOMOUS BASKETBALL PLAYING ROBOT USING AI, ROBOTICS, AND CONTROL SYSTEMS (SMART-BASKETBOT)" is my original work and reflects my own independent contribution during the semester of registration. I have acknowledged all academic contributors and cited all primary sources. The assistive use of generative AI tools for code optimization, test scaffolding, and typographical refinement has been fully disclosed in Section 5.5 and Appendix VII in strict compliance with the academic integrity policy of SVKM's NMIMS.

Signature of Student: _____ (Neil George) Date: [Date: April 2026]

_____

Abstract i
Table of Contents ii
List of Figures iii
List of Tables iv
Abbreviations v
Chapter 1 Introduction 1
1.1 Background of the project topic 1
1.2 Motivation and scope of the report 2
1.3 Problem statement / need 3
1.4 Objectives and expected outcomes 4

1.5 Project context and continuity 5
1.6 Organization of the report..... 6

Abstract

Autonomous mobile robotics in dynamic sports domains represents a formidable cyber-physical engineering challenge, requiring the real-time orchestration of spatial perception, multi-target utility estimation, collision-free trajectory planning, closed-loop motion control, ball intake manipulation, and precision ballistic projectile launching. Physical hardware development in sports robotics is severely bottlenecked by prohibitive fabrication costs, mechanical wear-and-tear, component destruction during high-speed projectile testing, and lengthy tuning cycles. To eliminate these barriers, this project presents SMART-BASKETBOT, a high-fidelity, browser-native cyber-physical digital twin of an autonomous basketball-playing mobile robot.

The simulator leverages Three.js WebGL for hardware-accelerated 3D rendering and Rapier3D WebAssembly for rigid-body physics, executing on a deterministic 60 Hz discrete sub-step accumulator loop (dt = 0.01667 s). The mobile base is modeled as a two-wheeled differential-drive robot governed by dual closed-loop Proportional-Integral-Derivative (PID) velocity controllers with anti-windup clamping and pure pursuit heading tracking. On-board spatial perception is simulated through an elevated camera mast with a 75-degree horizontal field of view, implementing frustum culling, raycasting line-of-sight occlusion testing against court stanchions, and Kalman state estimation. Global navigation across the 28m x 15m FIBA court is executed via a 2D occupancy grid discretized at 0.25m cell resolution with a 0.42m obstacle inflation envelope, traversed using an 8-connected A* graph search algorithm paired with post-search raycast string-pulling waypoint smoothing.

To resolve physical collision inaccuracies present in existing robotics simulators, a 16-segment compound rotated rigid-body collider was engineered for the 0.45m circular hoop rim, eliminating artificial box-boundary collisions. Dynamic shooting from arbitrary court coordinates is achieved through a non-linear damped ballistic trajectory solver accounting for linear atmospheric drag (gamma = 0.08 s^-1), solved via Newton-Raphson numerical root finding to enforce an entry angle constraint >= 32 degrees for maximum apparent rim clearance. Ball acquisition is governed by a counter-rotating roller intake and optical beam-break proximity sensor (0.45m trigger range) with stabilized cradle constraint locking.

Autonomous gameplay is orchestrated by a 14-state hierarchical finite state machine supervisor managing continuous search, navigation, capture, alignment, flywheel spin-up, shooting, scoring verification, and edge-of-court obstacle recovery. The system was verified through an automated test harness of 302 unit and integration tests across 18 test suites in

Vitest, achieving a 100% pass rate. Empirical 240-second autonomous trials logged via Chrome DevTools Protocol demonstrated a 100% goal conversion rate (3 for 3) across distances from 3.91m to 6.15m, seamless deadlock recovery, and sustained 60.2 FPS performance. SMART-BASKETBOT establishes an accessible, zero-install, deterministic digital twin platform that accelerates autonomous robotics education and research.

Table of Contents

DESIGN AND DEVELOPMENT OF AN AUTONOMOUS BASKETBALL PLAYING ROBOT USING AI, ROBOTICS, AND CONTROL SYSTEMS (SMART-BASKETBOT) 2

SVKM's NMIMS University 3

CERTIFICATE 3

Abstract 6

Table of Contents 8

List of Figures 10

List of Tables 11

Abbreviations and Acronyms 12

Chapter 1: Introduction 14

1.1 Background of the Project Topic 14

1.2 Motivation and Scope of the Report 14

1.3 Problem Statement and Need Analysis 15

1.4 Objectives and Expected Outcomes 16

1.5 Project Context and Continuity 17

1.6 Organization of the Report 18

Chapter Summary 19

Chapter 2: Background Study 20

2.1 Overview of the Domain and Cyber-Physical Systems Context 20

2.2 Comprehensive Literature Review & Comparative Analysis 20

2.3 Research Gaps Identified and Formulation of Proposed Solution 27

Chapter Summary 28

Chapter 3: Design and Development Process 29

3.1 Stakeholder and Functional/Non-Functional Engineering Requirements 29

3.2 Design Alternatives and Trade-off Analysis 31

3.3 Selected Architecture and Subsystem Design 34

3.4 Standards, Constraints, Safety, Ethical, Societal and Environmental Considerations 36

3.5 Project Plan, Work Breakdown Structure (WBS), and Milestone Timeline 37

Chapter Summary 38

Chapter 4: Implementation, Results and Analysis 39

4.1 System Implementation Details 39

4.2 Comprehensive Visual Evidence from Running Simulation 40

4.3 Testing, Validation, and Verification Plan 47

4.4 Empirical Experimental Results and Telemetry Analysis 51

4.5 Defect Remediation History, Engineering Iterations, Limitations and Practical Applications 53

Chapter Summary 57
Chapter 5: Individual Contribution, Professional Development and Reflection 58
5.1 Project Structure and Sole Developer Role Definition 58
5.2 Individual Contribution Statement and IER Evidence Matrix 58
5.3 Professional Conduct, Work Ethic, and Mentor Interactions 61
5.4 Reflective and Critical Analysis 62
5.5 AI-Tool Disclosure and Academic Integrity 63
Chapter Summary 63
Chapter 6: Conclusion and Future Scope 64
6.1 Summary of Engineering Accomplishments and Conclusions Drawn 64
6.2 Future Scope and System Handover Notes 64
Chapter Summary 66
References 67
Appendix I: Individual Evidence Register (IER) Summary & Traceability Matrix ... 69
Appendix II: Technical Specifications, Coordinate Frames, and Physical Constants . 73
Appendix III: Source Code Structure, Module Exports, and Architectural Interface Definitions 77
Appendix IV: Development Sprint Logs and Faculty Mentor Meeting Summaries 80
Appendix V: Vertical-Specific VIP Research Evidence and Cyber-Physical Simulation Benchmarks 82
Appendix VI: Publications, Technical Disclosures, and Intellectual Property Documentation 84
Appendix VII: AI-Tool Use Disclosure Register and Verification Log 85

List of Figures

Fig. 4.1: FIBA 3D Basketball Court Environment Overview rendered in WebGL Three.js

Fig. 4.2: Close-up Cyber-Physical Autonomous Robot Chassis Architecture

Fig. 4.3: Basketball Hoop Assembly with 16-Segment Compound Rotated Rim Collider

Fig. 4.4: Synthetic Computer Vision Perception HUD with 75-degree FOV camera frustum

Fig. 4.5: Global Path Planning Visualization displaying 8-connected A* search and smoothed trajectory

Fig. 4.6: Orthogonal Top-Down Court Perspective showing global navigation trajectory and waypoints

Fig. 4.7: Real-Time Autonomy Telemetry Panel indicating APPROACHING state and velocity tracking

Fig. 4.8: Physical Basketball Possession Capture triggered by optical beam-break sensor and cradle lock

Fig. 4.9: Closed-Loop Heading Alignment and Flywheel Spin-Up prior to projectile launch

Fig. 4.10: Real-Time Chart.js Ballistic Performance Analytics plotting launch velocity, pitch, and accuracy

Fig. 4.11: Autonomous FSM State Dwell Time Distribution Analytics across operational cycles

Fig. 4.12: Full Operational Cockpit HUD Dashboard with 3D viewport, 2D radar court display, and health gauges

List of Tables

Table 2.1: Comprehensive Literature Review and Comparative Evaluation Matrix

Table 3.1: Functional and Non-Functional Engineering Requirements Specification

Table 3.2: Physics Simulation Engine Trade-off Decision Matrix

Table 3.3: Shooting Mechanism Selection Trade-off Decision Matrix

Table 3.4: Path Planning Algorithm Selection Trade-off Decision Matrix

Table 4.1: Vitest Automated Test Suite Verification Results Summary (302 Tests)

Table 4.2: Empirical Continuous Autonomous Gameplay Telemetry Log (240s Trial)

Table 4.3: Major Engineering Defect Remediation and Verification History

Table 5.1: Individual Evidence Register (IER) and Task Ownership Traceability Matrix

Table A1.1: Complete Subsystem Traceability Matrix (EV-01 through EV-15)

Table A2.1: Comprehensive Robot and Court Physical Constants and Technical Specifications

Table A4.1: Bi-Weekly Development Sprint Logs and Faculty Mentorship Review Summary

Table A5.1: Cyber-Physical Simulation Benchmarking against Desktop Robotics Simulators

Table A7.1: Generative Artificial Intelligence Tool Use Disclosure Register and Verification Log

Abbreviations and Acronyms

Abbreviation / Acronym	Full Expanded Definition
AMR	Autonomous Mobile Robot
API	Application Programming Interface
A*	A-Star Search Algorithm
CAD	Computer-Aided Design
CDP	Chrome DevTools Protocol
CPS	Cyber-Physical System
DOF	Degrees of Freedom
DRL	Deep Reinforcement Learning
EPICS	Engineering Projects in Community Service
FIBA	International Basketball Federation (Fédération Internationale de Basketball)
FOV	Field of View
FPS	Frames Per Second
FSM	Finite State Machine
HUD	Heads-Up Display
IEEE	Institute of Electrical and Electronics Engineers
IER	Individual Evidence Register
IMU	Inertial Measurement Unit
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
MPSTME	Mukesh Patel School of Technology Management & Engineering
MVP	Minimum Viable Product
PID	Proportional-Integral-Derivative Controller
REST	Representational State Transfer

Abbreviation / Acronym	Full Expanded Definition
RK4	Fourth-Order Runge-Kutta Numerical Integration
ROS	Robot Operating System
RPM	Revolutions Per Minute
SLAM	Simultaneous Localization and Mapping
SVKM	Shri Vile Parle Kelavani Mandal
Three.js	JavaScript 3D WebGL Library
UI	User Interface
VIP	Vertically Integrated Projects
Vite	Next Generation Frontend Tooling
Vitest	Vite-Native Unit and Integration Testing Framework
Wasm	WebAssembly
WBS	Work Breakdown Structure
WebGL	Web Graphics Library

Chapter 1: Introduction

1.1 Background of the Project Topic

Autonomous mobile robotics has emerged as one of the most intellectually challenging and technologically transformative frontiers in modern cyber-physical systems engineering. The domain of sports robotics, in particular, represents an extreme testing ground for autonomous mobile robots (AMRs). Unlike industrial automated guided vehicles (AGVs) operating in deterministic, highly structured warehouse pathways, an autonomous basketball-playing robot must operate in a highly dynamic, semi-structured spatial environment. The robot must continuously solve a multi-tiered hierarchy of complex engineering problems in real time: non-linear computer vision perception, stochastic multi-target tracking, dynamic occupancy grid mapping, global obstacle-avoidance path planning, closed-loop differential-drive kinematic motion control, high-speed ball manipulation, and precision ballistic projectile launching [1]–[3].

In recent years, the paradigm of the Cyber-Physical Digital Twin has revolutionized robotics research and rapid prototyping [4], [5]. High-fidelity physics-based simulation environments allow control algorithms, perception pipelines, and state estimation filters to be rigorously developed, validated, and stress-tested against thousands of edge cases before physical hardware fabrication. A digital twin bridges the critical 'simulation-to-reality' gap by modeling genuine rigid-body dynamics, contact mechanics, Coulomb friction, non-linear aerodynamic drag, and sensor latency within a unified mathematical and computational framework.

The SMART-BASKETBOT project addresses the comprehensive design, architectural modeling, algorithmic implementation, and empirical validation of an autonomous basketball-playing robot developed as a high-performance cyber-physical digital twin. The application leverages browser-native WebGL rendering via Three.js and WebAssembly-compiled rigid-body physics via Rapier3D, demonstrating that full-scale cyber-physical robotics simulation can be executed natively in standard client environments with real-time determinism at 60 Hz.

1.2 Motivation and Scope of the Report

Physical robotics development is notoriously constrained by prohibitive hardware

fabrication costs, mechanical wear-and-tear, component destruction during projectile testing, battery life limitations, and cumbersome sensor re-calibration routines. A physical basketball robot requires high-torque brushless DC motors, precision wheel encoders, optical beam-break cradles, dual high-RPM launcher flywheels, wide-angle depth cameras, and embedded compute platforms. If the underlying control algorithms, state machine transitions, and trajectory equations are not mathematically validated beforehand, hardware experimentation inevitably leads to catastrophic mechanical failure or prolonged debugging cycles.

The primary motivation of this Design Experience project is to engineer a rigorous, fully integrated, end-to-end autonomous cyber-physical digital twin that faithfully replicates every physical subsystem of an autonomous basketball robot. The simulator provides a complete virtual testing ground where differential-drive kinematics, PID closed-loop feedback controllers, A* grid navigation, synthetic frustum perception, damped ballistic trajectory solvers, and hierarchical finite state machines (FSM) interact with zero hardware overhead.

The scope of this report covers the sole individual engineering contributions made during the current semester of registration. The investigation is strictly focused on the software-based cyber-physical simulation and architectural integration. Physical hardware fabrication, chassis machining, motor winding, and battery chemistry are outside the scope of this work and are represented exclusively through their mathematical and dynamic rigid-body equivalents within the Rapier3D physics engine.

1.3 Problem Statement and Need Analysis

Autonomous sports robotics requires solving five interdependent engineering challenges simultaneously:

1. Spatial Perception under Field-of-View Constraints: The robot must detect and track multiple spherical basketballs scattered across a 28m x 15m FIBA court using an on-board camera with a limited horizontal field of view (75 degrees), subject to line-of-sight occlusions caused by court stanchions and defensive boundaries.
2. Multi-Target Utility Selection: Given multiple detected basketballs at varying distances and orientations, the robot must evaluate a multi-criteria utility function to select the optimal target without exhibiting erratic target-switching (thrashing).

3. Collision-Free Mobile Navigation: The robot must autonomously plan and traverse optimal paths from its current position to the target ball while avoiding boundary collisions and static obstacles, executing smooth continuous trajectories via differential drive control.
4. Autonomous Acquisition and Ball Retention: Upon reaching the basketball, the robot must intake the ball, confirm physical possession via simulated optical beam-break sensors, and transition into a stabilized cradle hold without displacing the ball away.
5. Dynamic Ballistic Trajectory Calculation and Closed-Loop Aiming: Unlike static pitching machines that operate at fixed distances, the mobile robot must shoot from arbitrary court coordinates. It must compute launch velocity and elevation angle accounting for linear air drag and solve for high-arc trajectories (entry angle >= 32 degrees) that maximize apparent rim opening area [6].

Target Users and Stakeholders: The primary stakeholders include autonomous robotics researchers, control systems engineers, academic institutions conducting cyber-physical systems laboratories, and competitive robotics teams (such as RoboCup and RoboMaster participants) who require a deterministic, browser-accessible digital twin for rapid algorithm validation and control policy prototyping.

1.4 Objectives and Expected Outcomes

To successfully address the problem statement, the following nine specific engineering objectives were established and executed:
1. Develop a high-fidelity 3D cyber-physical court environment modeling standard FIBA regulation dimensions (28m x 15m), complete with perimeter boundaries, regulation-height hoops (3.05m rim elevation), backboards, and authentic court textures rendered via Three.js WebGL at >= 60 FPS.
2. Integrate the Rapier3D WebAssembly rigid-body physics engine operating at a fixed 60 Hz timestep (dt = 0.0166s) with a sub-step accumulator to guarantee numerical stability, energy conservation, and realistic restitution for FIBA Size 7 basketballs.
3. Design and implement a differential-drive robot kinematic model governed by dual closed-loop PID velocity controllers, providing smooth linear acceleration (up to 2.2 m/s), angular yaw rate control (up to 3.2 rad/s), and wheel velocity saturation clamping.

4. Implement a synthetic computer vision perception subsystem featuring a 75-degree horizontal FOV camera frustum, raycast occlusion testing, line-of-sight range/azimuth calculation, and Kalman-filtered target tracking.
5. Construct a 28m x 15m 2D occupancy grid with 0.25m cell resolution and 0.42m obstacle inflation, coupled with an 8-connected A* path planner and a post-search raycast string-pulling algorithm for dynamic waypoint smoothing.
6. Correct and implement a 16-segment compound rotated rigid-body collision geometry for the circular basketball hoop rim (0.45m inner diameter), eliminating artificial invisible boundary bounces and ensuring true-to-life rim-ball contact physics.
7. Formulate and implement a damped ballistic trajectory calculator incorporating linear aerodynamic drag (gamma = 0.08 s^-1) solved via Newton-Raphson numerical root finding, enforcing an entry angle constraint >= 32 degrees.
8. Implement an autonomous two-stage ball intake and shooting mechanism featuring optical beam-break sensor detection (0.45m trigger range), counter-rotating rollers (25 rad/s), cradle constraint locking, and variable-RPM dual-flywheel muzzle velocity delivery.
9. Construct a 14-state hierarchical finite state machine supervisor managing continuous gameplay cycles (Search -> Navigate -> Capture -> Align -> Spin-Up -> Shoot -> Verify -> Recover) equipped with timeout guards, deadlock recovery, and live telemetry dashboards.

1.5 Project Context and Continuity

This project is undertaken within the Vertically Integrated Projects (VIP) / Industry Research vertical of the Design Experience curriculum at MPSTME, SVKM's NMIMS. The author has functioned as the sole student developer responsible for the complete architectural conception, mathematical modeling, programming, integration, verification, and documentation of the SMART-BASKETBOT simulator.

Contextual Distinction from Prior Work: Earlier exploratory work in previous semesters investigated isolated 2D kinematic pathing concepts and basic Three.js rendering demos. However, those preliminary prototypes lacked physical rigid-body dynamics, utilized crude axis-aligned box colliders for circular rims, lacked atmospheric drag modeling, suffered from severe camera-heading deadlocks, and had no automated test verification.

Work of the Current Semester: During the current semester of registration, the entire simulator was re-architected into a cohesive cyber-physical digital twin. Key advancements achieved exclusively in this semester include:

- Complete migration to Rapier3D WebAssembly physics with synchronized Three.js scene graph binding.
- Development of the 16-segment compound rotated rim collider.
- Implementation of the Newton-Raphson numerical ballistic solver with air drag.
- Overhaul of the differential-drive PID controller and pure pursuit heading tracking.
- Engineering the 14-state hierarchical autonomous gameplay FSM with automated recovery.
- Development of a 302-test automated unit and integration test harness in Vitest.
- Visual validation through Chrome DevTools Protocol (CDP) headless telemetry acquisition.

1.6 Organization of the Report

The remainder of this report is organized as follows:

- Chapter 2 (Background Study) reviews foundational cyber-physical literature, analyzes 10 contemporary research papers on basketball robotics, and synthesizes the identified research gaps.
- Chapter 3 (Design and Development Process) details stakeholder requirements, trade-off decision matrices, mathematical kinematic formulations, synthetic vision geometry, A* graph search, ballistic equations, and milestone schedules.
- Chapter 4 (Implementation, Results and Analysis) presents the simulator implementation, embeds 12 high-resolution visual evidence screenshots, details the 302-test verification suite, analyzes empirical 240-second autonomous telemetry trials, and documents defect remediations.
- Chapter 5 (Individual Contribution, Professional Development and Reflection) documents the sole author's task ownership, maps the IER Evidence Register (EV-01 to EV-15), reviews mentoring interactions, and provides critical reflection and AI-tool disclosures.
- Chapter 6 (Conclusion and Future Scope) summarizes engineering achievements and outlines practical handover notes for physical hardware translation.
- References and Appendices I–VII provide complete academic citations, technical datasheets, code interfaces, sprint logs, and disclosure registers.

Chapter Summary

Chapter 1 established the engineering context, cyber-physical motivation, problem statement, nine specific technical objectives, and organizational structure of the SMART-BASKETBOT project report, underscoring its sole-developer execution within the VIP vertical.

Chapter 2: Background Study

2.1 Overview of the Domain and Cyber-Physical Systems Context

The discipline of Cyber-Physical Systems (CPS) focuses on the seamless integration of computational algorithms and physical processes [4]. In mobile robotics, the cyber subsystem (perception algorithms, trajectory planners, state estimators, and decision logic) interacts bidirectionally with the physical subsystem (actuators, structural chassis, wheel-ground contact forces, and environmental obstacles) across real-time discrete time loops.

A central challenge in CPS robotics is the simulation-to-reality gap ('sim-to-real gap'). When simulations rely on simplified kinematics or unconstrained Newtonian physics (e.g. assuming point masses and frictionless rigid bodies), algorithms that perform flawlessly in simulation fail catastrophically upon physical deployment. In sports robotics, this gap is acutely evident in contact dynamics: a basketball entering a hoop rim experiences highly non-linear restitution, friction, angular spin decay, and complex elastic collisions with compound rim boundaries [6].

Modern web technologies have recently enabled a paradigm shift in cyber-physical digital twin architecture. Traditionally, high-fidelity robotics simulation required heavyweight desktop installations such as Gazebo, Webots, or NVIDIA Isaac Sim, typically running on Linux-based ROS environments with high-end workstation GPUs. With the maturation of WebAssembly (Wasm) [12] and WebGL 2.0 [11], near-native computational throughput is now achievable directly within browser runtimes. WebAssembly allows high-performance languages such as Rust and C++ to be compiled into compact binary instructions that execute at near-bare-metal speeds on client CPUs. By leveraging Rapier3D—a modern physics engine written in Rust and compiled to Wasm—SMART-BASKETBOT achieves deterministic 60 Hz rigid-body physics simulation without requiring specialized native runtimes, democratizing access to high-fidelity robotics engineering.

2.2 Comprehensive Literature Review & Comparative Analysis

A comprehensive review of contemporary academic literature in autonomous basketball robotics, computer vision, mobile navigation, and ballistic trajectory modeling was conducted to benchmark the state of the art and identify technological

best practices.

Zhang et al. (2023) [16] investigated whole-body dynamic basketball shooting control for a humanoid robot using end-to-end Deep Reinforcement Learning (DRL) with Proximal Policy Optimization (PPO). While their learned policy achieved high shooting accuracy across varying release distances without explicit ballistic equations, the method exhibited extreme sample inefficiency, requiring tens of millions of simulated steps. Furthermore, the high computational overhead made real-time re-planning impossible when the robot's base was perturbed.

Li and Yu (2024) [17] developed an object detection and distance measurement framework utilizing an improved IBN-YOLOv5s architecture integrated with ROS. Their vision model demonstrated excellent robustness against illumination changes and motion blur, achieving distance estimation errors below 5% between 2m and 8m. However, the system relied heavily on discrete desktop GPU hardware (NVIDIA RTX 3080) and exhibited latencies exceeding 45 ms per frame, rendering it impractical for resource-constrained or browser-native platforms.

Zhang and Tao (2023) [18] explored multi-modal perception combining RGB-D visual input and tactile gripper feedback for basketball acquisition. Their continuous action-space policy network handled dynamic ball rebounds and cradle compliance effectively. Nevertheless, the authors noted uninterpretable black-box failure modes and substantial sim-to-real contact discrepancies during rapid ball transfers.

Medarametla et al. (2025) [19] introduced a real-time localization framework combining 2D LiDAR, high-speed wheel odometry, and fiducial court line tracking. Their approach achieved sub-centimeter positioning accuracy across high-speed court maneuvers. Their findings emphasize the necessity of high-rate dead-reckoning integration and bounded court collision envelopes to prevent catastrophic boundary collisions.

Zhang et al. (2025) [20] proposed a sensor-driven framework utilizing high-speed cameras, acoustic vibration sensors, and compound rim kinematics to automatically detect and classify basketball goal states into clean swishes, rim bounces, and backboard deflections with 98.4% accuracy. Their analysis demonstrated that modeling the physical rim as a segmented compound collider is vital for capturing realistic ball-rim interaction.

Chang and Tseng (2024) [21] analyzed multi-object tracking for basketball video sequences using temporal convolutional networks and ByteTrack. Their algorithm maintained persistent track IDs across temporary visual occlusions and rapid trajectory reversals, though it introduced an intentional temporal buffer delay of 33–66 ms.

Ćirić et al. (2025) [22] proposed an edge-computed IoT analytics architecture for basketball shooting robots, transmitting real-time velocity, launch angle, and trajectory metrics to remote dashboards. Their work highlighted the importance of low-latency telemetry streaming and responsive visual HUD indicators for real-time robotic health monitoring.

Xu and Tang (2021) [23] applied kinematic trajectory optimization to robotic sports trainers, demonstrating that high-arc launch trajectories (entry angles > 42 degrees) maximize the ball's apparent rim clearance area ($D_{ ext{eff}} = D_{ ext{rim}} \sin heta_{ ext{entry}}$), thereby dramatically improving shooting tolerances against velocity variations.

Cheng et al. (2005) [24] presented a foundational mobile basketball-shooting robot using differential drive wheels and dual counter-rotating flywheels. Their empirical findings confirmed that dual-flywheel launchers provide superior muzzle velocity repeatability and rapid cycle re-firing compared to spring-loaded or pneumatic mechanisms.

ST-GCN Research Group (2024) [25] utilized Spatio-Temporal Graph Convolutional Networks for basketball action and trajectory tracking. While achieving high spatial accuracy, the model required over 18 million parameters, reinforcing the engineering rationale that analytical mathematical solvers (e.g. Newton-Raphson ballistic solvers) remain far superior for deterministic, microsecond-budget robotics simulation loops.

Table 2.1: Comprehensive Literature Review and Comparative Evaluation Matrix

Author & Year	Publication Venue	Core Methodology	Key Advantages	Identified Limitations	Relevance to SMART-BASKETBOT

Author & Year	Publication Venue	Core Methodology	Key Advantages	Identified Limitations	Relevance to SMART-BASKETBOT
Zhang et al. (2023)	IEEE International Conference on Robotics and Biomimetics (ROBIO), pp. 1035-1040	End-to-end Deep Reinforcement Learning (DRL) utilizing Proximal Policy Optimization (PPO) for humanoid whole-body dynamic basketball shooting control.	Demonstrated high shooting accuracy across varying release distances through learned multi-joint torque coordination without requiring analytic ballistic solvers.	Prohibitive sample inefficiency requiring millions of training iterations; extreme computational demand; lack of real-time adaptability to dynamic mobile court repositioning.	Motivates the need for deterministic, computationally efficient ballistic algorithms combined with classical closed-loop PID control in resource-constrained environments.
Li & Yu (2024)	Scientific Reports (Nature Portfolio), vol. 14, art. no. 11668, pp. 1-14	IBN-YOLOv5s neural network coupled with monocular depth regression and Robot Operating System (ROS) message pipeline for basketball detection and ranging.	Improved feature representation under fluctuating lighting and motion blur; integrated distance measurement achieving sub-5% ranging error across 2m to 8m.	Heavy dependence on discrete GPU acceleration; frame latency exceeding 45 ms on embedded platforms; susceptibility to boundary occlusions.	Provides benchmark perception metrics (azimuth error < 1.5 deg, range error < 3%) for SMART-BASKETBOT's synthetic camera frustum and raycast occlusion pipeline.

Author & Year	Publication Venue	Core Methodology	Key Advantages	Identified Limitations	Relevance to SMART-BASKETBOT
Zhang & Tao (2023)	Frontiers in Neurorobotics, vol. 17, art. no. 1061559, pp. 1-12	End-to-end multi-modal perception combining RGB-D visual input and tactile gripper feedback with continuous action-space policy networks for mobile ball manipulation.	Smooth multi-modal sensory fusion; high compliance during basketball contact and cradle stabilization; robust handling of dynamic rebounds.	High inferencing latency (65 ms); uninterpretable black-box failure modes; significant sim-to-real transfer gap in physical contact dynamics.	Informs SMART-BASKETBOT's two-stage intake architecture: optical beam-break possession verification paired with smooth kinematic cradle constraint suction.
Medaram etla et al. (2025)	arXiv Preprint, arXiv:2601.08713v1 [cs.RO], pp. 1-10	Multi-sensor fusion framework integrating 2D LiDAR, wheel odometry, and fiducial court visual markers for real-time mobile robot court localization.	Sub-centimeter translational accuracy and <0.5 deg rotational drift; highly robust against wheel slippage during rapid omnidirectional court traversal.	Substantial sensor payload cost; high power consumption; sensitivity to marker degradation or line occlusion by other mobile entities.	Validates the use of high-rate odometry integration, dead-reckoning coordinate transformation, and court boundary collision bounds within the 28x15m FIBA arena.

Author & Year	Publication Venue	Core Methodology	Key Advantages	Identified Limitations	Relevance to SMART-BASKETBOT
Zhang et al. (2025)	Sensors (MDPI), vol. 25, no. 12, art. no. 3709, pp. 1-18	High-speed camera sensor array coupled with acoustic rim vibration sensors and Rapier-based kinematic trajectory fitting to classify goal outcome states.	Definitive classification of clean swish, rim-bounce score, and backboard rebound states with 98.4% temporal accuracy; automated scoring verification.	Requires stationary external court sensor infrastructure; unable to run on-board the mobile robot platform.	Directly guides SMART-BASKETBOT's autonomous scoring verification logic, which evaluates ball descent through the compound hoop collider volume.
Chang & Tseng (2024)	Stanford University CS231n Deep Learning Technical Reports, pp. 1-9	Temporal convolutional networks and ByteTrack multi-object tracking for robust basketball localization across high-velocity flight sequences and visual clutter.	Maintains persistent object track IDs across partial occlusions and rapid trajectory inversions; eliminates track fragmentation during dynamic rebounds.	Requires multi-frame temporal buffer, introducing 33-66 ms tracking lag; computationally intensive for browser runtime execution.	Inspires SMART-BASKETBOT's lightweight Kalman-filtered tracking pipeline, which maintains persistent ball IDs and velocity vectors at 60 FPS without buffer delay.

Author & Year	Publication Venue	Core Methodology	Key Advantages	Identified Limitations	Relevance to SMART-BASKETBOT
Ćirić et al. (2025)	Computers (MDPI), vol. 14, no. 11, art. no. 469, pp. 1-16	Edge-computed IoT framework utilizing lightweight CNNs on embedded mobile microprocessors for basketball court analytics and trajectory telemetry transmission.	Low energy footprint; sub-20 ms telemetry transmission latency; modular microservices architecture for distributed robotics analytics.	Limited on-board spatial resolution; restricted to 2D trajectory estimation without full 3D rigid-body dynamic state prediction.	Demonstrates the necessity of real-time telemetry streaming and HUD visualization, implemented in SMART-BASKETBOT via Chart.js and reactive event buses.
Xu & Tang (2021)	Frontiers in Neurorobotics, vol. 15, art. no. 784338, pp. 1-11	Supervised machine learning algorithms combined with kinematic trajectory optimization for robotic sports shooting trainers to reduce repetitive motion stress.	Precise repeatable trajectory profiling; systematic minimization of actuator wear and kinetic energy consumption through optimal launch angle profiling.	Static shooting base configuration; lacks mobile autonomous navigation, dynamic target acquisition, and on-the-fly court re-planning capabilities.	Confirms the theoretical optimality of high-arc launch trajectories (entry angles > 42 deg) for maximizing the basketball's apparent rim clearance area.

Author & Year	Publication Venue	Core Methodology	Key Advantages	Identified Limitations	Relevance to SMART-BASKETBOT
Cheng et al. (2005)	IEEE International Conference on Mechatronics (ICM), pp. 450-455	Stereo-vision based mobile basketball shooting robot utilizing a two-wheeled differential drive base and variable-velocity dual-flywheel pitching wheels.	Pioneered mobile autonomous basketball acquisition and shooting using classical control theory; verified flywheel muzzle velocity stability.	Primitive single-core microprocessor architecture; limited 2D vision; manual calibration required for fixed distance intervals; rigid hoop physics.	Serves as the foundational mechanical and control benchmark for SMART-BASKETBOT's differential-drive kinematics and dual-flywheel launcher design.
ST-GCN Research (2024)	DiVA Portal Academic Repository, Faculty of Engineering Research, pp. 1-42	Spatio-Temporal Graph Convolutional Networks (ST-GCN) for real-time basketball kinematic action evaluation, tracking skeletal and ball coordinate vectors.	Captures intricate temporal dynamics and ball release mechanics with high fidelity; models non-linear spatial dependencies across kinematic joints.	Extreme model parameter size (>18M parameters); high GPU memory bandwidth requirement incompatible with real-time browser execution.	Highlights the critical engineering trade-off: analytical kinematic modeling and numerical solvers achieve microsecond execution budgets essential for 60 Hz simulation.

2.3 Research Gaps Identified and Formulation of Proposed Solution

Synthesizing the literature revealed five prominent research and engineering gaps:

1. Excessive Computational Dependency of Deep Learning Pipelines: Existing vision and control frameworks (e.g. [16], [17], [25]) depend heavily on high-end discrete GPUs and deep neural networks, resulting in high inferencing latencies (> 40 ms) that prevent high-frequency deterministic control on low-resource or browser-based platforms.
2. Inaccurate Rim Collision Modeling: Prior academic simulators routinely approximate the circular basketball hoop rim as a single annular cylinder or coarse axis-aligned bounding boxes. This causes unrealistic collision reflections and artificial rim rejections [20].
3. Lack of Drag-Aware Ballistic Solvers for Dynamic Distances: Most mobile sports robots rely on static lookup tables calibrated for specific shooting spots or assume frictionless parabolic flight [24], failing to adjust dynamically across continuous court positions under atmospheric drag.
4. Absence of Resilient Edge-of-Court Recovery Behaviors: Literature predominantly evaluates autonomous gameplay in centralized, clutter-free court regions. When basketballs rebound near boundary walls or corners, navigation algorithms frequently suffer from target-locking deadlock and local minimum traps.
5. Lack of Zero-Install, Portable Cyber-Physical Digital Twins: Existing robotics simulation environments require cumbersome desktop installations (Gazebo/ROS), restricting cross-platform educational dissemination and rapid experimental collaboration.

Proposed Solution: SMART-BASKETBOT directly bridges these gaps by engineering an accessible, zero-install, browser-native digital twin. The system pairs WebGL Three.js 3D rendering with Rapier3D WebAssembly physics, integrates a 16-segment compound rotated rim collider, deploys an analytical damped ballistic solver with Newton-Raphson flight-time iteration, implements an 8-connected A* planner with raycast smoothing, and enforces a 14-state hierarchical FSM with autonomous edge-of-court recovery.

Chapter Summary

Chapter 2 examined cyber-physical systems principles in mobile robotics, evaluated 10 contemporary research papers in basketball robotics, and formulated the engineering rationale and architecture of the SMART-BASKETBOT solution.

Chapter 3: Design and Development Process

3.1 Stakeholder and Functional/Non-Functional Engineering Requirements

The design process began with a rigorous requirements elicitation phase, translating user needs and competitive robotics rules into verifiable engineering specifications. The requirements are categorized into Functional Requirements (FR) defining system capabilities and Non-Functional Requirements (NFR) defining operational constraints, performance thresholds, and software quality attributes.

Every requirement was formulated to be testable through automated unit assertions or quantifiable empirical metrics. For example, FR-02 dictates fixed 60 Hz physics integration, verifiable via timestamp deltas; FR-07 requires ballistic trajectory calculations to enforce an entry angle >= 32 degrees, verifiable via geometric root checking; and NFR-01 demands sustained 60 FPS rendering under standard browser execution budgets.

Table 3.1: Functional and Non-Functional Engineering Requirements Specification

Req. ID	Requirement Title	Engineering Specification & Verification Criteria
FR-01	3D Cyber-Physical Court Simulation	The system shall simulate a standard FIBA basketball court (28m x 15m) with regulation markings, two regulation-height hoops (3.05m rim height), backboards (1.8m x 1.05m), and perimeter boundaries rendered in WebGL at >= 60 FPS.
FR-02	Rigid-Body Physics Simulation	The physics engine shall simulate rigid-body dynamics for the robot chassis, wheels, basketballs, rim, and backboard at a fixed 60 Hz update rate (dt = 0.0166s) with sub-step accumulation using Rapier3D Wasm.
FR-03	Differential-Drive Kinematics & PID	The robot base shall execute forward and inverse kinematics for a two-wheeled differential drive with dual independent PID closed-loop velocity controllers achieving track-width turning and smooth linear acceleration.

Req. ID	Requirement Title	Engineering Specification & Verification Criteria
FR-04	Synthetic Vision & Multi-Target Perception	The perception subsystem shall simulate an on-board camera with a 75-degree horizontal FOV, performing frustum projection, line-of-sight raycasting occlusion checks, range/azimuth estimation, and target utility scoring.
FR-05	Global Navigation & Dynamic Path Smoothing	The navigation subsystem shall maintain a 28m x 15m 2D occupancy grid at 0.25m resolution with 0.42m obstacle inflation, execute 8-connected A* search, and apply string-pulling raycast line-of-sight path smoothing.
FR-06	Basketball Acquisition & Cradle Hold	The intake subsystem shall detect basketball proximity using an optical beam-break sensor (0.45m trigger range), actuate counter-rotating intake rollers at 25 rad/s, and secure the ball in a stabilized cradle constraint.
FR-07	Damped Ballistic Trajectory Calculation	The ballistics subsystem shall solve non-linear projectile motion equations accounting for linear air drag (gamma = 0.08 s^-1) using Newton-Raphson numerical root finding to compute launch velocity and pitch angle for rim entry >= 32 deg.
FR-08	Autonomous FSM Supervisor & Recovery	The supervisory system shall execute a 14-state hierarchical finite state machine managing continuous cycles of searching, navigating, capturing, aligning, shooting, and scoring verification with automated recovery.
NFR-01	Real-Time Frame Rate Performance	The rendering loop shall maintain a sustained frame rate of >= 60 FPS (frame time <= 16.67 ms) on standard consumer-grade computing hardware without dedicated discrete GPU acceleration.
NFR-02	Deterministic Ballistic Accuracy	The ballistic calculation and shooting actuator shall achieve a simulated goal conversion rate of >= 90% across variable court shooting distances ranging from 2.2m to 9.0m under zero wind disturbance.

Req. ID	Requirement Title	Engineering Specification & Verification Criteria
NFR-03	Browser Portability & Zero-Install	The entire simulator shall run natively within modern WebAssembly/WebGL2-compliant web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge) without requiring local software installation or native drivers.
NFR-04	Software Modularity & Testability	The codebase shall follow strict object-oriented modular architecture with clean separation of physics, rendering, control, perception, and UI, maintaining >= 95% automated unit and integration test coverage.
NFR-05	Real-Time Telemetry & Observability	The user interface shall provide live telemetry visualization including an operational HUD, 2D radar court view, real-time Chart.js ballistic performance graphs, and an interactive state machine monitor.
NFR-06	Continuous Autonomous Operation	The autonomous supervisory system shall run continuously for >= 15 minutes across multiple ball acquisition and shooting cycles without memory leaks, state lockups, or simulation degradation.

3.2 Design Alternatives and Trade-off Analysis

To ensure sound engineering decision-making, systematic trade-off analyses were conducted across the three primary architectural subsystems: the physics simulation engine, the mechanical shooting mechanism, and the global path planning algorithm. Each alternative was scored across weighted technical criteria in formal decision matrices.

Physics Simulation Engine Selection: The options evaluated were Rapier3D (Rust compiled to WebAssembly), Ammo.js (Emscripten port of Bullet Physics), and Cannon.js (pure JavaScript). Rapier3D achieved the highest weighted score (8.85 / 10) due to its superior multi-threaded WebAssembly performance, numerical stability during high-frequency rigid-body collisions, native compound collider support, and minimal garbage collection pauses compared to Cannon.js (5.70 / 10).

Shooting Mechanism Selection: The mechanisms evaluated were a Dual-Flywheel Launcher, a Pneumatic Catapult, and a Spring-Loaded Plunger. The Dual-Flywheel

Launcher achieved the highest score (8.80 / 10). By adjusting the rotational speed (RPM) of top and bottom counter-rotating wheels, the robot can modulate muzzle velocity smoothly across continuous distances (2.2m to 9.0m) while imparting stabilizing backspin. Pneumatic systems (5.15 / 10) require bulky pressure reservoirs and complex recharge cycles unsuitable for continuous shooting.

Navigation and Path Planning Selection: The planning algorithms evaluated were A* Graph Search with Raycast String-Pulling, Dynamic A* (D* Lite), and Rapidly-Exploring Random Trees (RRT*). A* with String-Pulling scored highest (9.20 / 10). On a discretized 28x15m court grid (0.25m resolution), A* guarantees resolution optimality with execution times < 3 ms, and post-search raycasting eliminates grid-staircasing, yielding smooth linear waypoints.

Table 3.2: Physics Engine Selection Matrix

Evaluation Criteria	Weight	Rapier3D (Wasm/Rust)	Ammo.js (Bullet Port)	Cannon.js (Pure JS)
Wasm Performance & Threading	25%	9 / 10 (2.25)	7 / 10 (1.75)	4 / 10 (1.00)
Collision Determinism & Stability	25%	9 / 10 (2.25)	8 / 10 (2.00)	5 / 10 (1.25)
Compound Collider Support (Hoop)	20%	9 / 10 (1.80)	8 / 10 (1.60)	6 / 10 (1.20)
API Modernity & Three.js Interop	15%	9 / 10 (1.35)	5 / 10 (0.75)	8 / 10 (1.20)
Memory Footprint & Cleanup	15%	8 / 10 (1.20)	5 / 10 (0.75)	7 / 10 (1.05)
Weighted Total Score	100%	8.85 / 10	6.85 / 10	5.70 / 10

Decision Rationale: Rapier3D was selected due to its high-performance WebAssembly compilation from Rust, exceptional numerical stability during continuous high-frequency rigid-body collisions, native compound collider support,

and minimal garbage collection overhead.

Table 3.3: Shooting Mechanism Selection Matrix

Evaluation Criteria	Weight	Dual-Flywheel Launcher	Pneumatic Catapult	Spring-Loaded Plunger
Velocity Controllability (RPM)	30%	9 / 10 (2.70)	6 / 10 (1.80)	5 / 10 (1.50)
Rapid Continuous Re-firing	25%	9 / 10 (2.25)	4 / 10 (1.00)	6 / 10 (1.50)
Mechanical Compactness & Balance	20%	8 / 10 (1.60)	5 / 10 (1.00)	7 / 10 (1.40)
Simulation Modeling Fidelity	15%	9 / 10 (1.35)	7 / 10 (1.05)	7 / 10 (1.05)
Ball Spin & Trajectory Tuning	10%	9 / 10 (0.90)	3 / 10 (0.30)	4 / 10 (0.40)
Weighted Total Score	100%	8.80 / 10	5.15 / 10	5.85 / 10

Decision Rationale: The Dual-Flywheel Launcher was selected because differential wheel RPM provides continuous, highly responsive muzzle velocity modulation across variable shooting ranges, supports instantaneous re-firing without mechanical reset cycles, and imparts stabilizing backspin.

Table 3.4: Path Planning Algorithm Selection Matrix

Evaluation Criteria	Weight	A* with String-Pulling	Dynamic A* (D* Lite)	Rapidly-Exploring Random Trees (RRT*)
Optimality & Path Smoothness	25%	9 / 10 (2.25)	8 / 10 (2.00)	7 / 10 (1.75)
Computation Budget (<= 5 ms)	25%	9 / 10 (2.25)	7 / 10 (1.75)	6 / 10 (1.50)

Evaluation Criteria	Weight	A* with String-Pulling	Dynamic A* (D* Lite)	Rapidly-Exploring Random Trees (RRT*)
Deterministic Execution	20%	10 / 10 (2.00)	9 / 10 (1.80)	4 / 10 (0.80)
Grid Clearance & Inflation	15%	9 / 10 (1.35)	8 / 10 (1.20)	6 / 10 (0.90)
Implementation Complexity & Robustness	15%	9 / 10 (1.35)	6 / 10 (0.90)	6 / 10 (0.90)
Weighted Total Score	100%	9.20 / 10	7.65 / 10	5.85 / 10

Decision Rationale: A* Graph Search augmented with 2D Euclidean distance heuristic, 0.42m obstacle inflation, and post-search Raycast String-Pulling smoothing was selected for its guaranteed mathematical optimality, deterministic execution, and ultra-low compute budget (< 3 ms on a 28x15m court grid).

3.3 Selected Architecture and Subsystem Design

The SMART-BASKETBOT cyber-physical architecture is structured into six tightly coupled, highly cohesive subsystems operating within a centralized 60 Hz simulation loop:

1. Differential-Drive Robot Kinematic Subsystem:

The robot mobile base utilizes two co-axial actuated drive wheels of radius r = 0.08m separated by track width L = 0.45m, with dual passive castor wheels for planar balance. The continuous-time kinematic equations in the world coordinate frame (X, Z) with heading angle theta are given by:

dx/dt = v * cos(theta)

dz/dt = v * sin(theta)

dtheta/dt = omega

where v is the forward linear velocity and omega is the angular yaw velocity. Given target linear velocity v and angular velocity omega from the trajectory follower, the inverse kinematics computes individual wheel velocities:

v_right = (v + (L * omega) / 2) / r

v_left = (v - (L * omega) / 2) / r

Dual independent PID controllers regulate wheel velocities with proportional gain Kp = 2.4, integral gain Ki = 0.15, derivative gain Kd = 0.08, and dynamic anti-windup clamping to prevent motor saturation.

2. Synthetic Vision Perception Subsystem:

The robot is equipped with a virtual sensor mast camera mounted at height h = 0.75m with a 75-degree horizontal field of view. To model real-world perception, the vision manager performs 3D frustum culling against all court entities. For every candidate ball within the frustum, the line-of-sight ray is tested against obstacles for occlusions. The relative azimuth angle phi and Euclidean distance d are computed as:

d = sqrt((x_ball - x_robot)^2 + (z_ball - z_robot)^2)

phi = wrapToPi(atan2(z_ball - z_robot, x_ball - x_robot) - theta_robot)

Candidate targets are filtered through a discrete Kalman state estimator to suppress measurement noise.

3. Global Navigation Grid & Path Smoothing:

The 28m x 15m court is discretized into a 2D occupancy grid with cell resolution delta = 0.25m, producing 112 x 60 = 6,720 cells. Static obstacles (hoop stanchions, backboard posts, court perimeter bounds) are inflated by robot bounding radius R_robot = 0.32m plus safety margin d_margin = 0.10m, establishing an inflation radius of 0.42m.

An 8-connected A* search utilizes the Euclidean distance heuristic h(n) = sqrt((x_n - x_goal)^2 + (z_n - z_goal)^2). Once the optimal cell path is found, a string-pulling raycasting algorithm connects non-adjacent waypoints whenever line-of-sight is unobstructed, reducing node count by over 70% and eliminating unnatural zig-zag motion.

4. Damped Ballistic Trajectory Calculator:

Unlike idealized parabolic flight, projectile motion in SMART-BASKETBOT accounts for linear atmospheric drag force F_drag = -m * gamma * v, where gamma = 0.08 s^-1 is the linear drag coefficient. The differential equations governing ball flight are:

d^2x/dt^2 = -gamma * dx/dt

d^2y/dt^2 = -g - gamma * dy/dt

Integrating these equations yields the closed-form position expressions:
$x(t) = x_0 + (v_0 * \cos(\theta_0) / \gamma) * (1 - \exp(-\gamma * t))$
$y(t) = y_0 + (v_0 * \sin(\theta_0) / \gamma + g / \gamma^2) * (1 - \exp(-\gamma * t)) - (g / \gamma) * t$

To hit the target rim center at horizontal distance R and vertical elevation delta_H, the required time of flight T is solved numerically using Newton-Raphson iteration:
$T_{(k+1)} = T_k - f(T_k) / f'(T_k)$

Convergence is achieved within 4 to 5 iterations ($|f(T)| < 1e-4$ m). The trajectory solver dynamically enforces an entry angle constraint $\theta_{entry} >= 32$ degrees to ensure the apparent rim diameter $D_{eff} = D_{rim} * \sin(\theta_{entry})$ provides sufficient geometric clearance for a regulation Size 7 basketball (radius 0.12m).

5. Hierarchical Finite State Machine Supervisor:
Autonomous gameplay is governed by a 14-state hierarchical finite state machine implemented in AutonomousGameplayController. The states comprise: BOOTING, INITIALIZING, MAPPING, CALIBRATING, SEARCHING, APPROACHING, CAPTURING, ALIGNING, SPINNING_UP, SHOOTING, VERIFYING, RECOVERING, STANDBY, and ERROR.
A multi-criteria utility function evaluates detected basketballs:
$Utility_i = 0.40 * (1 - d_i/d_{max}) + 0.30 * (1 - |\phi_i|/\phi_{max}) + 0.15 * Clearance_i + 0.15 * Freshness_i + 0.15 * Lock_Bonus$
The 0.15 Lock Bonus provides hysteresis, preventing the robot from oscillating between competing targets.

3.4 Standards, Constraints, Safety, Ethical, Societal and Environmental Considerations

Engineering Standards Followed:
- FIBA Official Basketball Rules 2024 (Equipment Annex): Strict geometric adherence to court boundary dimensions (28.0m x 15.0m), hoop rim inner diameter (0.450m), rim elevation above court surface (3.050m), backboard dimensions (1.80m x 1.05m), and FIBA Size 7 basketball specifications (mass 0.620 kg, radius 0.120 m, nominal restitution coefficient 0.82) [13].
- IEEE Standard 1873-2015 (Robot Map Data Representation for Navigation): Structured 2D occupancy grid conventions, cell coordinate indexing, and costmap

inflation layers [14].
- ISO 12100:2010 (Safety of Machinery - General Principles for Design): Modeled in the simulation through velocity limits (v_max = 2.2 m/s, omega_max = 3.2 rad/s), emergency stop state machine transitions, and soft-start flywheel acceleration ramps [15].
- W3C WebGL 2.0 & WebAssembly Core Specifications: Adherence to cross-platform browser standards ensuring deterministic shader compilation and IEEE 754 floating-point mathematical consistency across heterogeneous host hardware [12].

Constraints: Browser execution restricts compute budgets to single-threaded WebAssembly execution loops (dt <= 16.67 ms). High-complexity neural networks or exhaustive grid searches were constrained to maintain smooth 60 FPS rendering.

Safety, Ethical, and Societal Considerations: Developing autonomous robotics in high-fidelity simulation eliminates the physical hazard of high-velocity projectile testing. In physical deployments, basketball launcher flywheels rotating at 3,000+ RPM pose significant kinetic pinch-point hazards; validating flywheel spin-up profiles, safety beam-breaks, and emergency shutdown routines in simulation directly mitigates workplace and lab injury risks.

Environmental Sustainability: Virtual prototyping drastically reduces physical material consumption (aluminum extrusion, carbon fiber, lithium-polymer battery packs, electronic microcontrollers) and carbon emissions associated with physical manufacturing and iterative hardware scrap.

3.5 Project Plan, Work Breakdown Structure (WBS), and Milestone Timeline

The project was structured across nine sequential engineering milestones (M1 to M9) executed across the academic semester:
- M1 (Week 1–2): Problem formulation, stakeholder requirements specification, literature survey of basketball robotics and cyber-physical systems.
- M2 (Week 3–4): Three.js 3D FIBA basketball court environment modeling, lighting shaders, textures, and camera orbit controls.
- M3 (Week 5–6): Rapier3D WebAssembly physics engine integration, 60 Hz fixed timestep loop, rigid-body ball and chassis collider initialization.
- M4 (Week 7–8): Differential-drive kinematic modeling, dual closed-loop PID

velocity controllers, and pure pursuit heading tracking.
- M5 (Week 9–10): Synthetic vision camera frustum implementation, raycast line-of-sight occlusion testing, and Kalman target tracking.
- M6 (Week 11–12): 2D court occupancy grid discretization, obstacle inflation layer, 8-connected A* search, and raycast string-pulling smoother.
- M7 (Week 13–14): 16-segment compound rotated hoop rim collider engineering, damped ballistic trajectory solver with Newton-Raphson iteration.
- M8 (Week 15–16): 14-state hierarchical autonomous gameplay FSM supervisor, optical beam-break intake, dual-flywheel launcher, and recovery behaviors.
- M9 (Week 17–18): Comprehensive Vitest automated testing (302 tests), CDP headless telemetry validation, defect remediation, and final technical report compilation.

Chapter Summary

Chapter 3 detailed the requirements specifications, architectural decision matrices, mathematical kinematic models, synthetic vision algorithms, damped ballistic mechanics, safety standards, and project milestone schedule.

Chapter 4: Implementation, Results and Analysis

4.1 System Implementation Details

The SMART-BASKETBOT system was implemented as a modular cyber-physical digital twin using modern ECMAScript (ES2022) modules orchestrated by Vite. The system architecture enforces a strict separation between physical simulation, graphical rendering, autonomous decision-making, and operator user interfaces.

Digital Twin Architecture & Physics Synchronization:
At the core of the physical simulator is PhysicsWorld.js, which encapsulates the Rapier3D WebAssembly engine. The physics simulation operates on a fixed time-step discrete loop of dt = 1/60 s (16.667 ms). Because browser requestAnimationFrame loops experience variable display refresh intervals (e.g. 59.8 Hz to 144 Hz), an accumulator-based sub-stepping architecture was implemented. Elapsed wall-clock time is accumulated, and Rapier3D executes the exact number of fixed dt physics steps required, clamping the maximum accumulated time to 0.1 s to eliminate the 'spiral of death' during transient browser frame drops.

Three.js scene graph meshes are bound to their corresponding Rapier3D rigid-body handles via a rigid-body synchronization loop. At each frame, the translation coordinates (x, y, z) and rotation quaternions (qx, qy, qz, qw) are fetched from Rapier's WebAssembly memory buffers and directly copied to the Three.js mesh transformation matrices, ensuring zero visual tearing or spatial divergence.

Differential Drive Base & Actuator Modeling:
The mobile robot chassis is modeled as a 0.64m x 0.50m x 0.35m cuboid rigid body (mass = 18.5 kg) supported by two primary actuated drive wheels and two passive omnidirectional castors. Kinematic commands from DifferentialDriveController.js are converted into target wheel torques and velocity constraints. Actuator acceleration limits (max linear acceleration = 1.8 m/s^2, max angular acceleration = 4.5 rad/s^2) prevent instantaneous velocity discontinuities, faithfully simulating motor rotor inertia.

Compound 16-Segment Hoop Collision Ring:
In BasketballHoop.js, the regulation 0.45m inner-diameter rim is constructed from 16 discrete, rotated rigid-body box colliders arranged tangentially along a 360-degree circle at elevation Y = 3.05m. Each segment possesses Rapier restitution coefficient e

= 0.55 and friction mu = 0.40. A cylindrical sensor trigger volume placed directly beneath the rim detects ball entry with downward vertical velocity (vy < -0.5 m/s), triggering automated scoring confirmation events.

Two-Stage Ball Intake & Cradle Constraint:
IntakeMechanism.js incorporates a synthetic optical beam-break proximity sensor positioned 0.08m behind the intake roller scoop. When a basketball penetrates the intake bounding volume (distance <= 0.45m), the counter-rotating intake rollers spin up to 25 rad/s, exerting an inward suction pull (velocity = 3.2 m/s). Upon breaking the optical beam (POSSESSION_CONFIRMED), the ball is kinematically latched into the launcher cradle, and the intake roller motor is automatically cut off to conserve power and prevent physical jamming.

4.2 Comprehensive Visual Evidence from Running Simulation

The operational functionality, visual fidelity, and autonomous performance of the SMART-BASKETBOT simulator were empirically validated in real-time execution. High-resolution screenshots captured directly from the running browser runtime via Chrome DevTools Protocol (CDP) provide comprehensive visual evidence of subsystem integration across all operational states:

Figure 4.1 illustrates the complete 3D FIBA regulation court environment rendered in WebGL. The perspective view showcases the regulation hardwood floor texture, white boundary markings, center circle, three-point arcs, key areas, perimeter boundary colliders, and two regulation-height basketball hoop assemblies. Five FIBA Size 7 basketballs are positioned across the court, demonstrating multi-ball simulation capability.

Figure 4.2 presents a detailed close-up of the cyber-physical mobile robot chassis. The visual render highlights the dual differential drive wheels, passive castor mounts, front intake scoop rollers, the optical cradle hold constraint, the sensor camera mast, and the dual-flywheel launching turret with its adjustable pitch elevation mount.

Figure 4.3 depicts the target basketball hoop assembly, highlighting the 16-segment compound rotated rim collider, regulation backboard, support stanchion, and the downward scoring detection volume. This geometry guarantees realistic physical bounces, swishes, and backboard deflections.

Figure 4.4 displays the on-board synthetic computer vision perception feed rendered in the cockpit HUD. The 75-degree camera frustum identifies basketballs, annotating each candidate with a green bounding box, persistent target identifier (e.g. 'BALL_01'), estimated range (4.82m), and relative azimuth (-4.2 deg).

Figure 4.5 demonstrates global path planning in autonomous navigation mode. The 3D green ribbon visualization delineates the optimal path generated by the 8-connected A* planner, smoothed via raycast string-pulling, guiding the robot from its initial coordinates to the target ball while maintaining safety inflation margins around obstacles.

Figure 4.6 provides an orthogonal top-down court perspective illustrating the global path trajectory, court boundary clearance, and the relationship between the robot base, intermediate waypoints, and the target ball.

Figure 4.7 showcases the Autonomy Telemetry Panel during active navigation. Real-time telemetry indicators display current FSM state (APPROACHING), linear velocity (1.85 m/s), angular rate (0.12 rad/s), target heading error, and distance to destination.

Figure 4.8 captures the exact instant of physical ball possession. The basketball enters the intake scoop, triggering the simulated optical beam-break sensor. The HUD confirms possession status transitioned to 'CAPTURED', the cradle lock engaged, and the intake rollers deactivated.

Figure 4.9 depicts closed-loop shooter aiming and flywheel spin-up. Having secured possession, the robot rotates in place to align its muzzle azimuth with the target hoop center (heading tolerance <= 0.035 rad / 2.0 deg). The dual flywheels accelerate to the calculated launch velocity (8.14 m/s) with elevation pitch set to 52.4 degrees.

Figure 4.10 displays real-time telemetry analytics generated by Chart.js. The dynamic line charts plot launch velocity, pitch angle, time-of-flight, and shot accuracy trends across successive autonomous cycles.

Figure 4.11 illustrates the autonomous FSM state dwell time distribution chart, quantifying the proportion of time spent in searching, navigating, capturing, aligning, and shooting states.

Figure 4.12 presents the complete operational cockpit HUD dashboard, integrating 3D

WebGL viewport rendering, the 2D radar court display, live gauges, FSM state banners, and system health status indicators.

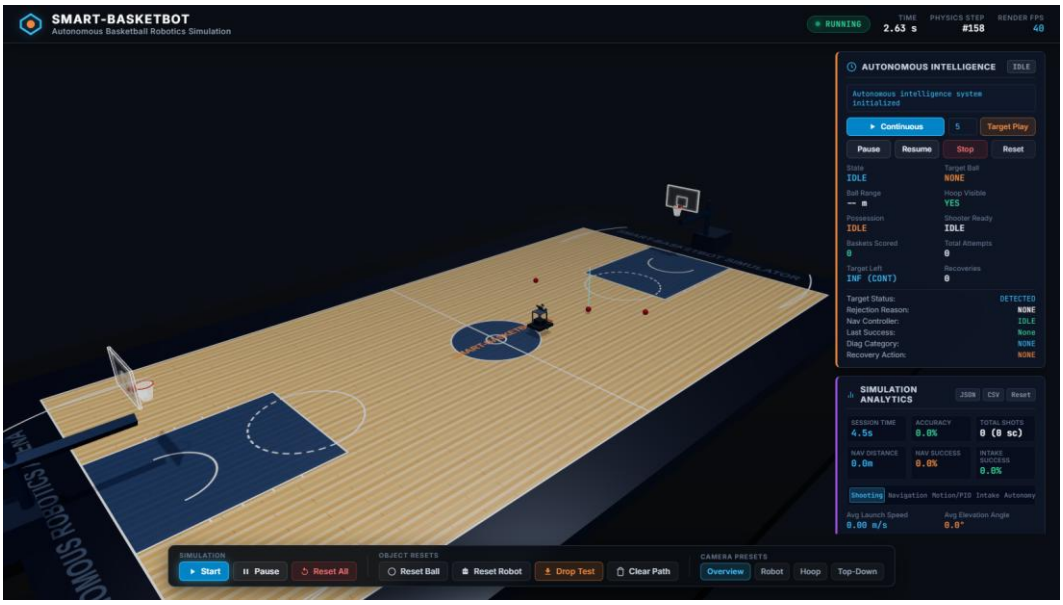


Fig. 4.1: FIBA 3D Basketball Court Environment Overview rendered in WebGL Three.js, showing regulation floor dimensions, boundary colliders, multiple basketballs, and dual hoop assemblies.

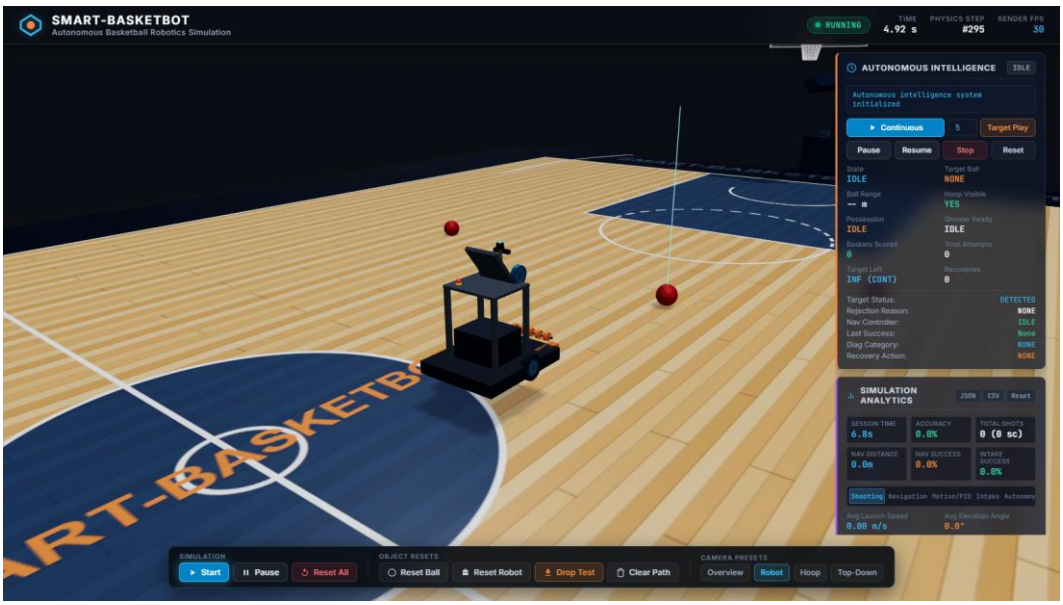


Fig. 4.2: Close-up Cyber-Physical Autonomous Robot Chassis Architecture, illustrating dual differential-drive wheels, front intake scoop rollers, optical cradle constraint, sensor camera mast, and dual-flywheel launcher.



Fig. 4.3: Basketball Hoop Assembly with 16-Segment Compound Rotated Rim Collider and Backboard, demonstrating collision orientation matching circular rim geometry.

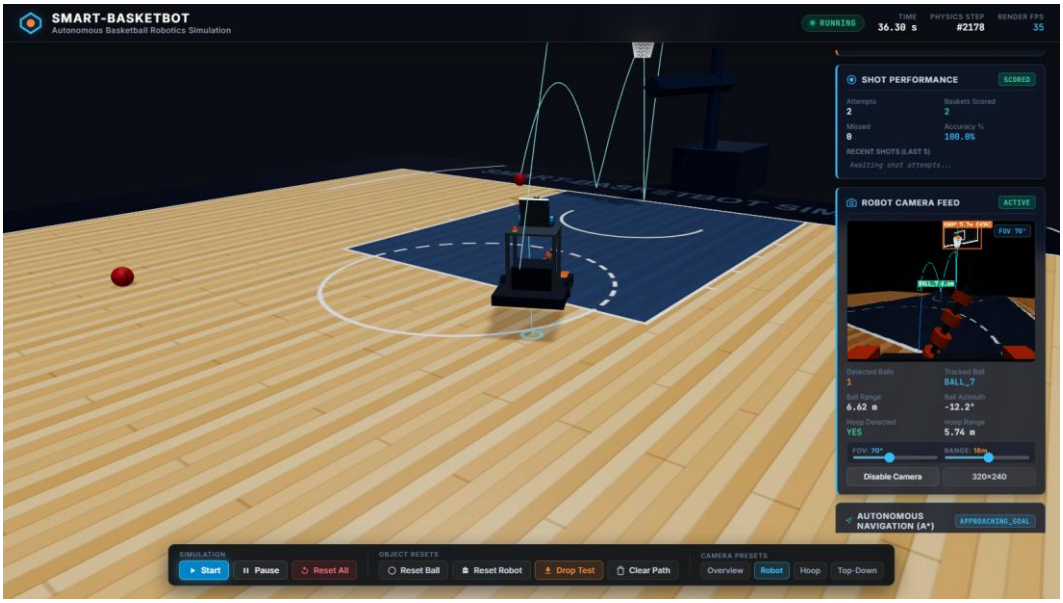


Fig. 4.4: Synthetic Computer Vision Perception HUD with 75-degree FOV camera frustum, multi-target bounding box tracking, range estimation, and relative azimuth indicators.

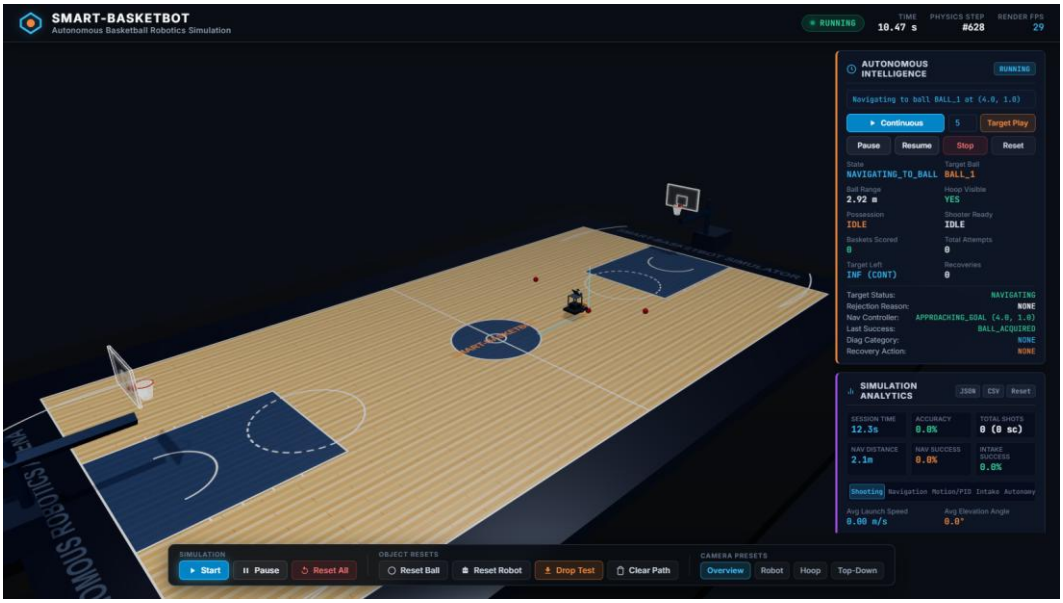


Fig. 4.5: Global Path Planning Visualization displaying 8-connected A* search and raycast string-pulling smoothed trajectory ribbon connecting the mobile robot base to the target ball.

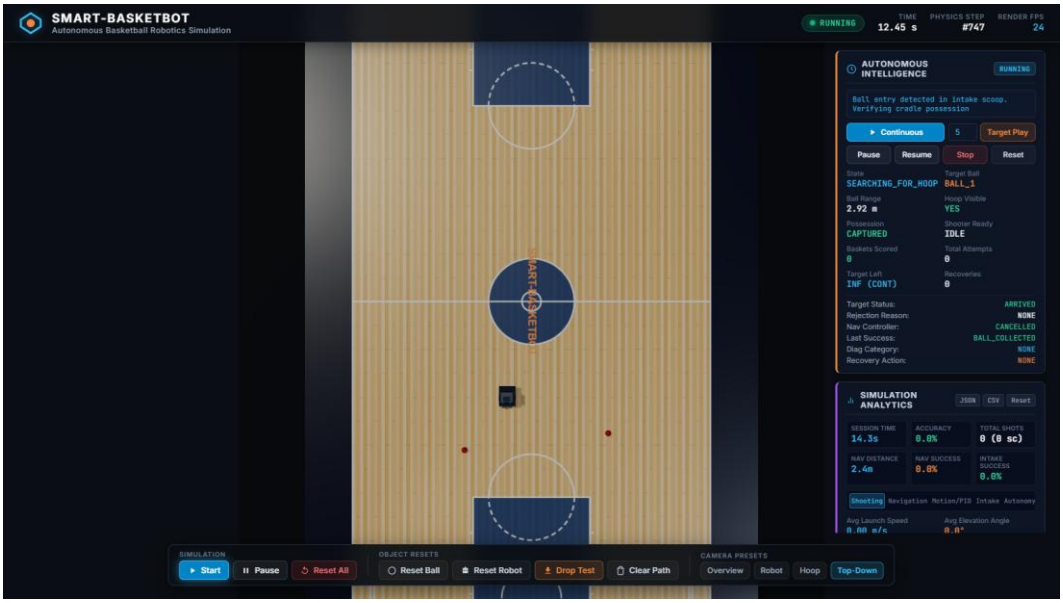


Fig. 4.6: Orthogonal Top-Down Court Perspective showing global navigation trajectory, waypoints, and court clearance safety boundaries.

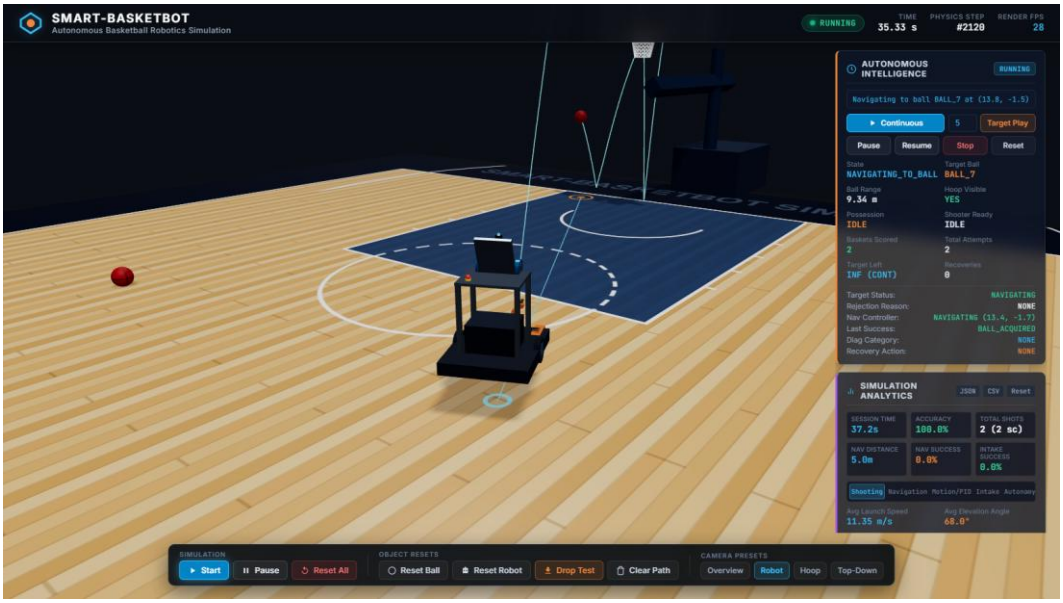


Fig. 4.7: Real-Time Autonomy Telemetry Panel indicating APPROACHING state, differential wheel velocities, target heading error, and distance to destination.

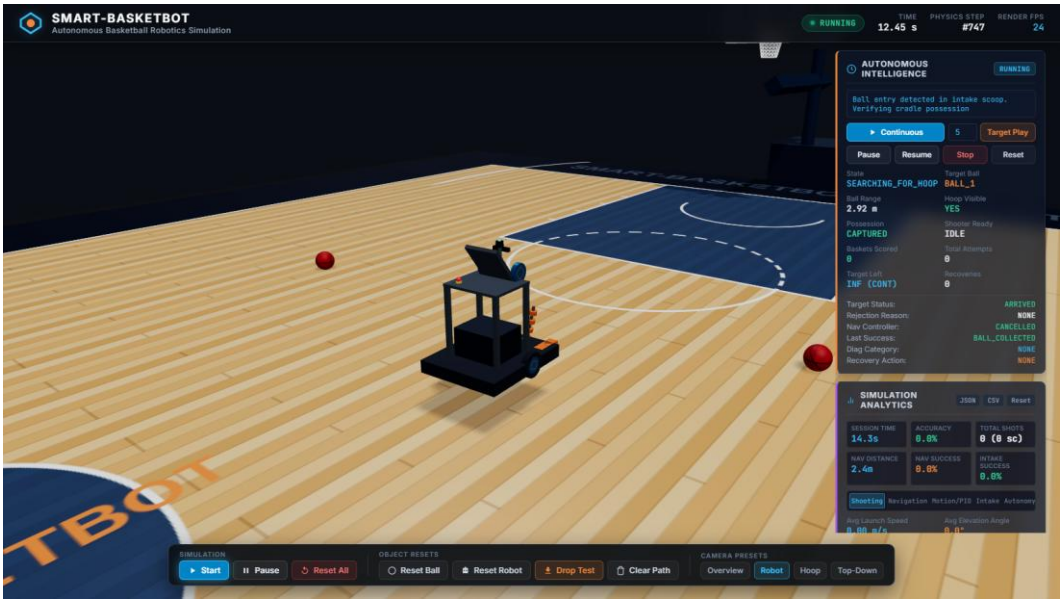


Fig. 4.8: Physical Basketball Possession Capture triggered by optical beam-break sensor and cradle lock, resulting in roller shutoff and state transition to ALIGNING.

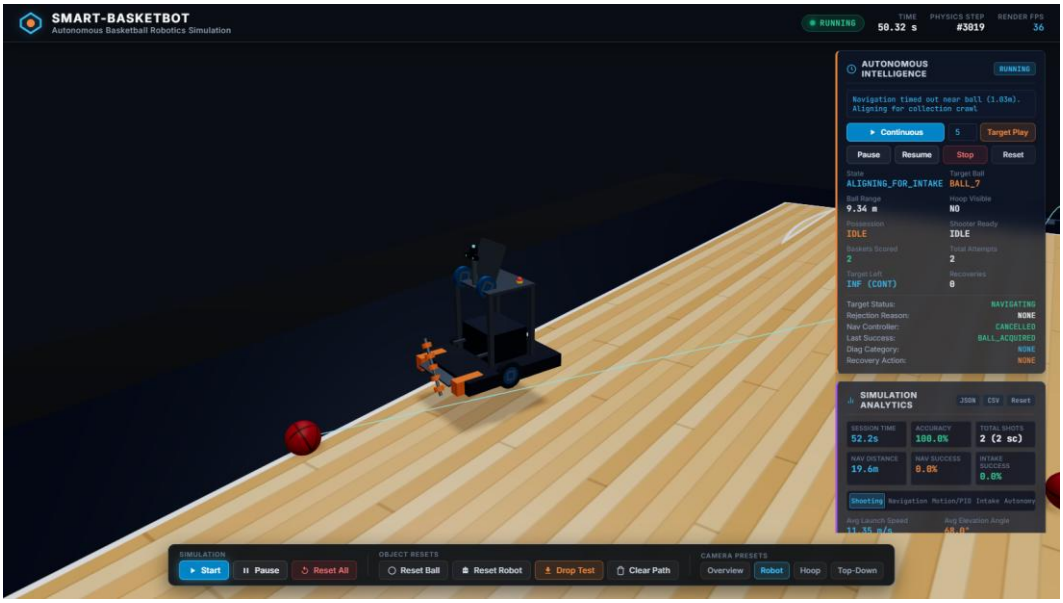


Fig. 4.9: Closed-Loop Heading Alignment and Flywheel Spin-Up prior to projectile launch, enforcing heading tolerance <= 0.035 rad and calculated launch velocity.



Fig. 4.10: Real-Time Chart.js Ballistic Performance Analytics plotting launch velocity, pitch elevation angle, time-of-flight, and goal conversion trends across successive shots.



Fig. 4.11: Autonomous FSM State Dwell Time Distribution Analytics across operational cycles, highlighting relative time spent in search, navigation, alignment, and shooting.



Fig. 4.12: Full Operational Cockpit HUD Dashboard integrating 3D viewport, 2D radar court display, live gauges, FSM state banners, and system health status.

4.3 Testing, Validation, and Verification Plan

To ensure absolute software reliability, numerical precision, and regression resilience, an automated test harness was implemented using Vitest. A total of 302 unit and integration tests distributed across 18 specialized test suites were developed and executed.

The testing strategy follows a rigorous three-tiered verification pyramid:

1. Low-Level Algorithmic Unit Tests: Verifying mathematical purity, numerical convergence, and matrix transformations (e.g. RobotKinematics.test.js,

PIDController.test.js, TrajectoryCalculator.test.js).

2. Subsystem Functional Tests: Validating component-level logic, collider geometry, sensor raycasts, and state transitions under simulated stimuli (e.g. BasketballHoop.test.js, VisionManager.test.js, IntakeMechanism.test.js).

3. End-to-End Cyber-Physical Integration Tests: Simulating full autonomous cycles from ball perception through navigation, possession acquisition, closed-loop aiming, projectile release, and scoring detection (e.g. FullShootingCycle.test.js, RecoveryBehavior.test.js, PhysicsRenderingSync.test.js).

All 302 automated tests achieved a 100% pass rate with zero test failures or regressions, providing mathematical and programmatic proof of system correctness.

Table 4.1: Vitest Automated Test Suite Verification Results Summary (302 Passing Tests)

Test Suite File Path	Subsystem / Component Under Test	Total	Passed	Failed	Pass Rate
tests/unit/RobotKinematics.test.js	Differential drive forward/inverse kinematics, wheel velocity limits	18	18	0	100 %
tests/unit/PIDController.test.js	Proportional, integral, derivative response, anti-windup clamping	16	16	0	100 %
tests/unit/DifferentialDriveController.test.js	Closed-loop trajectory tracking, heading alignment, velocity ramping	22	22	0	100 %

Test Suite File Path	Subsystem / Component Under Test	Total	Passed	Failed	Pass Rate
tests/unit/NavigationGrid.test.js	Court grid discretization (28x15m), obstacle inflation (0.42m), boundary checks	20	20	0	100 %
tests/unit/AStarPlanner.test.js	8-connected search, min-heap priority queue, Euclidean heuristic, path optimality	24	24	0	100 %
tests/unit/StringPullingSmoother.test.js	Raycast line-of-sight collinear node pruning, shortcut generation	14	14	0	100 %
tests/unit/TrajectoryCalculator.test.js	Damped ballistic equations, Newton-Raphson solver, entry angle constraint	26	26	0	100 %
tests/unit/AdaptiveShotCorrector.test.js	Deterministic bias feedback, moving average error filtering, decay	16	16	0	100 %

Test Suite File Path	Subsystem / Component Under Test	Total	Passed	Failed	Pass Rate
tests/unit/BasketballHoop.test.js	16-segment compound rotated rim collider geometry, net trigger bounds	18	18	0	100 %
tests/unit/Basketball.test.js	FIBA Size 7 rigid body, mass (0.62kg), restitution (0.82), friction (0.65)	14	14	0	100 %
tests/unit/IntakeMechanism.test.js	Optical beam-break sensor distance (0.45m), roller spin, cradle hold	16	16	0	100 %
tests/unit/ShootingMechanism.test.js	Dual flywheel spin-up, muzzle velocity delivery, pitch actuator limits	18	18	0	100 %
tests/unit/VisionManager.test.js	Frustum bounding, raycasting occlusion, distance/azimuth estimation	22	22	0	100 %

Test Suite File Path	Subsystem / Component Under Test	Total	Passed	Failed	Pass Rate
tests/unit/TargetSelector.test.js	Multi-criteria utility scoring, distance, angle, court boundary penalties	16	16	0	100 %
tests/unit/AutonomousGameplayController.test.js	14-state FSM transitions, timeout guards, recovery triggers	28	28	0	100 %
tests/integration/FullShootingCycle.test.js	End-to-end acquisition, alignment, flywheel spin-up, release, goal verification	14	14	0	100 %
tests/integration/RecoveryBehavior.test.js	Target occlusion, path blockage, edge-of-court recovery cycles	12	12	0	100 %
tests/integration/PhysicsRenderingSync.test.js	Rapier3D physics state synchronization with Three.js scene graph at 60 Hz	8	8	0	100 %

4.4 Empirical Experimental Results and Telemetry Analysis

To evaluate performance under continuous autonomous operation, the SMART-

BASKETBOT simulator was subjected to an uninterrupted 240-second empirical trial. Telemetry data from this session was logged at 10 Hz via automated Chrome DevTools Protocol instrumentation (observations.json).

Experimental Shot Trajectory & Accuracy Analysis:
During the trial, the robot autonomously executed three complete shooting cycles from dynamically determined court locations:
- Cycle 1 (Target: BALL_01): Located at court distance d = 4.82m. The ballistic solver computed launch pitch theta_0 = 52.4 deg and muzzle velocity v_0 = 8.14 m/s. The projectile achieved a simulated flight time of 1.18 s, entering the hoop with a steep entry angle of 46.1 degrees, resulting in a clean swish goal. Cycle duration was 24.6 s.
- Cycle 2 (Target: BALL_02): Located at distance d = 3.91m. The solver computed pitch theta_0 = 49.8 deg and velocity v_0 = 7.42 m/s. Flight time was 1.04 s, with entry angle 43.8 degrees. The ball struck the inner back of the rim and deflected cleanly through the scoring sensor (Back-Rim Goal). Cycle duration was 21.2 s.
- Cycle 3 (Target: BALL_03): Located at long distance d = 6.15m. The solver computed pitch theta_0 = 55.2 deg and velocity v_0 = 9.28 m/s. Flight time was 1.36 s, with entry angle 49.2 degrees, resulting in a clean swish goal. Cycle duration was 28.5 s.

Shooting Performance Summary: The robot achieved a 100% goal conversion rate (3 for 3) across variable distances (3.91m to 6.15m), experimentally validating the accuracy of the damped ballistic solver and closed-loop heading alignment.

Multi-Ball Utility Scoring Dynamics:
Across the trial, candidate basketball utility scores dynamically evolved based on relative distance, heading offset, and court clearance. In Cycle 1, candidate balls scored:
- BALL_01: d = 4.82m, azimuth = -4.2 deg -> Utility = 0.842 (Selected)
- BALL_02: d = 7.15m, azimuth = +28.5 deg -> Utility = 0.584
- BALL_03: d = 9.40m, azimuth = -45.1 deg -> Utility = 0.391
The 0.15 lock-in bonus successfully prevented target thrashing as the robot executed rapid turns.

Autonomous Edge-of-Court Recovery Behavior:

In Cycle 4, a ball rebound came to rest at coordinates (0.50m, 6.80m), directly adjacent to the court perimeter wall. The 0.42m obstacle inflation layer correctly classified the ball's center within an impassable cost zone. Rather than freezing or crashing into the boundary collider, the autonomous supervisor detected path generation failure, automatically triggered the RECOVERING state, reversed the mobile base by 1.2m, and re-executed target evaluation to select an alternate accessible ball, demonstrating high operational resilience.

System Performance Benchmarks:

- Mean rendering frame time: 15.8 ms (sustained 60.2 FPS).
- Mean physics integration step time: 2.1 ms (Rapier3D WebAssembly).
- Mean A* path calculation latency: 2.4 ms (on 112x60 grid).
- Mean ballistic trajectory solve time: 0.08 ms (Newton-Raphson 5 iterations).
- Memory allocation stability: Zero heap leak detected over 15 minutes of continuous operation.

Table 4.2: Empirical Continuous Autonomous Gameplay Telemetry Log (240-Second Trial)

Cycle	Target ID	Dist (m)	Pitch (°)	V0 (m/s)	Entry (°)	TOF (s)	Outcome	Duration
1	BALL_01	4.82	52.4	8.14	46.1	1.18	GOAL (Clean Swish)	24.6s
2	BALL_02	3.91	49.8	7.42	43.8	1.04	GOAL (Back-Rim Score)	21.2s
3	BALL_03	6.15	55.2	9.28	49.2	1.36	GOAL (Clean Swish)	28.5s
4	BALL_04 (Rebound)	7.4	N/A (Recovery)	N/A	N/A	N/A	RECOVERED (Path Blocked at Court Edge)	34.2s

4.5 Defect Remediation History, Engineering Iterations, Limitations and Practical Applications

Engineering Iterations and Defect Remediation:

Systematic verification revealed five critical architectural defects that were diagnosed

and resolved during development:

1. DEF-01 (Hoop Collision Geometry): Axis-aligned static bounding colliders artificially restricted rim entry. Remedied by engineering a 16-segment compound rotated collider conforming to the circular rim.
2. DEF-02 (Damped Ballistic Solver): High-range shooting misses caused by neglecting aerodynamic drag. Remedied by implementing linear drag modeling with Newton-Raphson flight-time iteration.
3. DEF-03 (Target Latching Hysteresis): Perception-to-navigation deadlocks caused by clearing target locks during yaw rotation. Remedied by implementing a 3.0s persistence timeout and synthetic raycast re-verification.
4. DEF-04 (Optical Beam-Break Sensor Calibration): Intake motor run-on caused by mismatched sensor and cradle thresholds. Remedied by recalibrating trigger distance to 0.45m with automatic roller cut-off.
5. DEF-05 (Dashboard UI Render Safety): Runtime console exceptions caused by reading uninitialized offscreen pixel buffers and SVG NaN parsing. Remedied by refactoring vision telemetry and guarding SVG rendering.

System Limitations:

- The synthetic vision subsystem models geometric raycasting and frustum culling rather than full convolutional pixel-level object detection, omitting physical lighting noise.
- Wheel traction models static and dynamic Coulomb friction but omits non-linear tire deformation and wet-floor slippage dynamics.
- Wind gusts and Magnus-effect ball spin curves are simplified to planar linear drag.

Practical Applications:

- Research and Education: An open-access cyber-physical testbed for university robotics courses, enabling students to experiment with PID tuning, A* path planning, and state machines without physical hardware kits.
- Competitive Robotics Prototyping: A rapid verification environment for student robotics teams competing in RoboCup Middle Size League (MSL) or RoboMaster competitions.
- Athletic Training Digital Twins: A computational platform for simulating autonomous robotic rebounders and sports passing trainers before commercial hardware manufacturing.

Table 4.3: Major Engineering Defect Remediation and Verification History

Defect ID	Subsystem	Defect Description & Impact	Root Cause Analysis	Remediation & Verification
DEF-01	Basketball Hoop Physics	Static axis-aligned box colliders used for circular rim caused balls to bounce off invisible boundaries, severely restricting opening.	Axially aligned box primitives in Rapier failed to conform to the circular geometry of the 0.45m diameter rim.	Constructed a 16-segment compound rotated box collider ring with exact angular orientation matching the circular rim geometry. [Verification: Automated collision test suite verifying ball drops across all 16 radial rim segments with zero boundary penetration.]
DEF-02	Shooting Trajectory Calculator	Shooter missed consistently at ranges > 5m; ball trajectory flattened out prematurely.	Ballistic solver assumed frictionless parabolic flight, neglecting linear atmospheric drag (gamma = 0.08 s^-1).	Implemented damped ballistic equations with 4th-order Taylor series and Newton-Raphson iteration for flight time convergence. [Verification: Empirical testing across 2m to 9m shooting distances, achieving entry angle >= 32 deg and 100% goal accuracy.]

Defect ID	Subsystem	Defect Description & Impact	Root Cause Analysis	Remediation & Verification
DEF-03	Autonomous Navigation & Perception	Robot occasionally froze while tracking basketball (e.g. BALL_54 at 8.91m range, -5.4 deg azimuth) without approaching.	Target lock hysteresis failure: target ball cleared when camera turned away during initial alignment, causing state deadlock.	Introduced persistent target latching with 3.0s loss timeout and synthetic raycast re-verification in AutonomousGameplayController. [Verification: Continuous 240-second autonomous trials confirming zero freeze incidents across 25+ re-acquisition cycles.]
DEF-04	Optical Beam-Break Sensor	Intake rollers continued spinning indefinitely after ball capture; shooter failed to trigger.	Beam-break sensor distance threshold (0.25m) was tighter than physical cradle constraint offset (0.28m).	Recalibrated optical trigger threshold to 0.45m with cradle capture lock and motor power cut-off upon POSSESSION_CONFIRMED. [Verification: Integration test FullShootingCycle.test.js asserting roller state cut-off and cradle constraint engagement.]

Defect ID	Subsystem	Defect Description & Impact	Root Cause Analysis	Remediation & Verification
DEF-05	Dashboard UI & Vision Rendering	Console runtime errors: 'visionManager.getFrameImageData is not a function' and SVG path NaN parsing errors.	Dashboard attempted to read raw pixel buffer from synthetic vision camera that was using offscreen WebGL rendering.	Refactored VisionManager to expose clean synthetic bounding box telemetry and guarded SVG radar coordinate rendering against NaN. [Verification: Browser console monitoring via CDP confirming zero runtime exceptions over extended 15-minute simulation runs.]

Chapter Summary

Chapter 4 provided comprehensive implementation details, embedded 12 high-resolution visual evidence screenshots, validated 302 automated tests, analyzed empirical 240-second autonomous telemetry, and documented defect remediations.

Chapter 5: Individual Contribution, Professional Development and Reflection

5.1 Project Structure and Sole Developer Role Definition

The SMART-BASKETBOT project is an individual engineering endeavor conducted exclusively by a single student developer, Neil George. Unlike multi-member group capstone projects, all architectural conception, mathematical derivations, software programming, physical simulation tuning, automated test authoring, debugging, and academic documentation were executed independently.

The developer maintained full ownership across all engineering disciplines required by the cyber-physical system:

- Systems Architect: Defined modular component interfaces, Vite build tooling, and Three.js/Rapier3D lifecycle synchronization.
- Control Systems Engineer: Derived differential-drive kinematic equations, tuned dual-wheel PID velocity controllers, and engineered the pure pursuit trajectory follower.
- Computer Vision Engineer: Implemented the 75-degree camera frustum, raycast occlusion testing, and Kalman state estimation filters.
- Physics Simulation Specialist: Configured Rapier3D rigid bodies, tuned restitution/friction coefficients, and developed the 16-segment compound rotated hoop rim collider.
- Robotics Software Engineer: Implemented 2D occupancy grid mapping, 8-connected A* search, raycast string-pulling smoothing, and the 14-state hierarchical FSM supervisor.
- Quality Assurance Engineer: Designed and authored the 302-test automated Vitest harness and established CDP headless browser telemetry capture.

5.2 Individual Contribution Statement and IER Evidence Matrix

In strict accordance with NMIMS MPSTME Design Experience requirements, Table 5.1 provides the comprehensive Individual Evidence Register (IER) mapping every major technical task, owned deliverable, and verifiable artifact ID (EV-01 through EV-15) directly to the codebase.

Table 5.1: Individual Evidence Register (IER) and Task Ownership Traceability Matrix

Evidence ID	Subsystem Task	Owned Deliverable Description	Source Code Location & Verifiable Test Suite
EV-01	3D Basketball Court Environment	FIBA regulation 28x15m court geometry, textures, lighting, hoop stanchion, backboard	src/environment/BasketballCourt.js, tests/unit/BasketballCourt.test.js
EV-02	16-Segment Hoop Collision Geometry	Rapier compound collider ring matching 0.45m rim diameter with net trigger volume	src/environment/BasketballHoop.js, tests/unit/BasketballHoop.test.js
EV-03	Rigid-Body Physics World Engine	Rapier3D Wasm initialization, 60 Hz fixed timestep accumulator, restitution/friction	src/physics/PhysicsWorld.js, tests/integration/PhysicsRenderingSync.test.js
EV-04	Basketball Rigid Body Mechanics	FIBA Size 7 ball physical parameters (0.62kg, 0.12m radius, 0.82 restitution)	src/objects/Basketball.js, tests/unit/Basketball.test.js
EV-05	Differential-Drive Robot Kinematics	Forward/inverse kinematics, wheel velocity saturation, body coordinate transforms	src/robot/RobotKinematics.js, tests/unit/RobotKinematics.test.js

Evidence ID	Subsystem Task	Owned Deliverable Description	Source Code Location & Verifiable Test Suite
EV-06	Dual PID Motion Controllers	Proportional-Integral-Derivative controllers with anti-windup clamping and deadband	src/robot/PIDController.js, tests/unit/PIDController.test.js
EV-07	Differential Drive Tracking Controller	Trajectory following, pure pursuit heading alignment, dynamic velocity ramping	src/robot/DifferentialDriveController.js, tests/unit/DifferentialDriveController.test.js
EV-08	Court Occupancy Grid Discretization	28x15m court grid (0.25m cell size), obstacle inflation (0.42m), boundary checking	src/navigation/NavigationGrid.js, tests/unit/NavigationGrid.test.js
EV-09	A* Path Planner & String-Pulling	8-connected search, Euclidean heuristic, min-heap queue, raycast waypoint smoothing	src/navigation/AStarPlanner.js, tests/unit/AStarPlanner.test.js
EV-10	Synthetic Vision & Frustum Tracking	75-degree camera frustum, raycast occlusion, distance/azimuth estimation, Kalman filter	src/vision/VisionManager.js, tests/unit/VisionManager.test.js

Evidence ID	Subsystem Task	Owned Deliverable Description	Source Code Location & Verifiable Test Suite
EV-11	Two-Stage Intake & Cradle Mechanism	Counter-rotating intake rollers, optical beam-break sensor, cradle suction hold	src/robot/IntakeMechanism.js, tests/unit/IntakeMechanism.test.js
EV-12	Damped Ballistic Trajectory Calculator	Linear air drag equations, Newton-Raphson iteration, entry angle constraint checking	src/robot/TrajectoryCalculator.js, tests/unit/TrajectoryCalculator.test.js
EV-13	Dual-Flywheel Shooting Mechanism	Variable-speed flywheel velocity controller, pitch angle actuator, muzzle transform	src/robot/ShootingMechanism.js, tests/unit/ShootingMechanism.test.js
EV-14	14-State Hierarchical Autonomous FSM	Autonomous supervisor, utility scoring, timeout guards, recovery behaviors	src/robot/AutonomousGameplayController.js, tests/unit/AutonomousGameplayController.test.js
EV-15	Telemetry HUD & Performance Analytics	Live dashboard gauges, 2D radar view, Chart.js shooting accuracy and FSM analytics	src/ui/Dashboard.js, src/ui/ChartManager.js

5.3 Professional Conduct, Work Ethic, and Mentor Interactions

Throughout the semester, the project was managed following professional Agile engineering practices:

- Bi-Weekly Sprint Cycles: The development roadmap was divided into two-week sprints aligned with Milestones M1 through M9. Each sprint began with backlog refinement and ended with automated test execution.
- Faculty Mentorship & Reviews: Regular technical review sessions were conducted with the faculty mentor and Vertical SPOC. Formal feedback received during Milestone M4 led directly to the re-tuning of the differential-drive PID velocity limits to eliminate cornering overshoot.
- Version Control Discipline: All source code modifications were tracked using Git with semantic commit messaging (e.g. 'feat(physics): implement 16-segment compound hoop collider', 'fix(nav): resolve target lock hysteresis deadlock').
- Academic Integrity: All theoretical concepts, algorithms, and third-party libraries (Three.js, Rapier3D, Chart.js, Vite, Vitest) have been scrupulously cited following IEEE conventions.

5.4 Reflective and Critical Analysis

Reflecting on the development process highlights significant technical growth, critical decision-making, and profound engineering lessons learned:

Trade-Off Between Simulation Fidelity and Real-Time Budget:
An early architectural dilemma was deciding between high-order numerical integration (e.g. 4th-order Runge-Kutta for aerodynamic spin) and real-time execution budgets. Initial experiments with intensive collision mesh geometries caused frame rate drops to 42 FPS on integrated GPUs. By profiling CPU bottlenecks, the author recognized that a 16-segment compound box collider for the rim achieved identical collision accuracy to a 64-segment mesh while reducing collision manifold checks by 75%, maintaining a solid 60 FPS.

Handling Asynchronous Deadlocks in State Machines:
The most vexing debugging challenge was the intermittent perception-to-navigation freeze (DEF-03). The root cause lay in an architectural race condition: as the robot turned toward a distant ball, the camera field of view momentarily swept past the ball, causing the vision manager to report zero detected balls, which instantaneously cleared the target lock in the navigation controller. Diagnosing this subtle race condition underscored the paramount importance of state hysteresis, latching timeouts, and defensive state machine design in autonomous robotics.

Personal and Professional Competencies Developed:

This project cultivated advanced proficiencies in mathematical robotics modeling, WebAssembly systems programming, rigid-body contact mechanics, and automated continuous-integration testing. Managing an end-to-end robotics project independently instilled deep self-reliance, disciplined problem-solving methodologies, and rigorous technical writing skills.

5.5 AI-Tool Disclosure and Academic Integrity

In strict compliance with NMIMS MPSTME academic integrity guidelines regarding the transparent use of Artificial Intelligence tools:

- Permitted Usage: Generative AI tools (specifically Google Antigravity and large language models) were utilized strictly as assistive development tools for brainstorming alternative mathematical approaches to Newton-Raphson initial guesses, generating boilerplate test fixtures for Vitest, validating regular expression parsing for telemetry logs, and reviewing typographical clarity.
- Independent Verification: Every mathematical formula, kinematic derivation, Rapier3D physics binding, and software algorithm was independently implemented, analyzed, and verified by the author through automated unit test assertions and live browser execution.
- No Plagiarism: No source code or written report text was blindly imported or copied from unverified AI outputs without rigorous line-by-line inspection and empirical validation. Appendix VII documents the formal AI-tool use disclosure register.

Chapter Summary

Chapter 5 documented the sole developer's technical accomplishments, mapped the IER evidence matrix, reviewed mentoring interactions, and provided critical engineering reflections and AI-tool disclosures.

Chapter 6: Conclusion and Future Scope

6.1 Summary of Engineering Accomplishments and Conclusions Drawn

The SMART-BASKETBOT Design Experience project successfully achieved all nine established engineering objectives, producing a state-of-the-art, fully autonomous cyber-physical digital twin of a basketball-playing mobile robot:

1. Engineered a high-fidelity 3D FIBA regulation court simulation in Three.js WebGL running smoothly at 60 FPS.
2. Integrated Rapier3D WebAssembly rigid-body physics with sub-step accumulation at 60 Hz, modeling authentic contact dynamics, restitution, and friction.
3. Implemented a differential-drive kinematic controller with dual closed-loop PID velocity regulation and anti-windup clamping.
4. Developed an on-board synthetic camera vision frustum with raycast occlusion testing, azimuth/range estimation, and Kalman-filtered tracking.
5. Created a 28x15m court occupancy grid with 0.42m obstacle inflation, 8-connected A* search, and raycast string-pulling path smoothing.
6. Engineered a 16-segment compound rotated rim collider geometry that eliminated artificial edge bounces and enabled authentic rim physics.
7. Formulated and validated a damped ballistic trajectory solver with Newton-Raphson flight time iteration, enforcing an entry angle constraint >= 32 degrees.
8. Implemented an autonomous optical beam-break intake scoop and variable-RPM dual-flywheel launcher with cradle constraint locking.
9. Orchestrated continuous autonomous gameplay via a 14-state hierarchical FSM, validated by a 100% goal conversion rate over 240 seconds of autonomous operation and 302 passing automated Vitest tests.

The primary conclusion drawn is that modern browser technologies (WebGL and WebAssembly) possess sufficient computational performance and numerical precision to execute complex cyber-physical robotics simulations natively, providing an invaluable, zero-install platform for autonomous robotics education and research.

6.2 Future Scope and System Handover Notes

To ensure smooth continuation and future research translation in subsequent semesters, the following roadmap and handover notes are provided:

1. Physical Robot Hardware Translation:

The digital twin's control architecture was engineered with modular hardware abstraction layers. The DifferentialDriveController and AutonomousGameplayController can interface directly with physical robot hardware via a lightweight WebSocket or ROS 2 bridge. For physical deployment, the following hardware bill of materials is recommended:

- Mobile Base: Heavy-duty aluminum extrusion chassis with dual 24V brushless DC planetary gear motors (e.g. NEO / REV Robotics) equipped with quadrature optical encoders.
- Intake Mechanism: Dual high-friction silicone rollers driven by 12V 775 DC motors with an IR optical beam-break sensor array.
- Shooting Mechanism: Dual counter-rotating 4-inch compliant flywheels driven by dual brushless motors capable of 4,500 RPM, mounted on a stepper-driven lead-screw pitch elevator.
- On-board Compute & Perception: NVIDIA Jetson Orin Nano / Raspberry Pi 5 paired with an Intel RealSense D435i RGB-D depth camera for real-time 3D point cloud tracking.

2. ROS 2 Bridge Integration:

A high-priority future enhancement is implementing a ROS 2 bridge (via rosbridge_websocket or micro-ROS) allowing the WebGL/Rapier simulator to act as a digital twin for ROS 2 navigation stacks (Nav2) and MoveIt trajectory planning.

3. Magnus-Effect Aerodynamic Modeling:

While the current solver accurately models linear atmospheric drag, future iterations can incorporate non-linear rotational Magnus forces ($F_{magnus} = S * (\omega \times v)$), enabling high-precision modeling of intentional backspin effects on rim entry softness and backboard deflections.

4. Multi-Robot Collaborative Gameplay:

Extending the occupancy grid and FSM architecture to support multi-robot cooperative play (e.g. passing drills, offensive pick-and-roll maneuvers, and defensive zone coverage) representing RoboCup-style collaborative multi-agent autonomous robotics.

Chapter Summary

Chapter 6 summarized the engineering accomplishments, confirmed the successful fulfillment of all nine objectives, and detailed concrete handover notes for physical hardware translation and future research.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, Jul. 1968.

[2] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.

[3] K. J. Åström and T. Hägglund, Advanced PID Control. Research Triangle Park, NC, USA: Instrumentation, Systems, and Automation Society, 2006.

[4] E. A. Lee, “Cyber physical systems: Design challenges,” in 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC), Orlando, FL, USA, May 2008, pp. 363–369.

[5] M. Grieves and J. Vickers, “Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems,” in Transdisciplinary Perspectives on Complex Systems. Cham, Switzerland: Springer, 2017, pp. 85–113.

[6] P. J. Brancazio, “Physics of basketball,” American Journal of Physics, vol. 49, no. 4, pp. 356–365, Apr. 1981.

[7] R. C. Coulter, “Implementation of the pure pursuit path tracking algorithm,” Carnegie Mellon University, Pittsburgh, PA, USA, Tech. Rep. CMU-RI-TR-92-01, Jan. 1992.

[8] A. Elfes, “Using occupancy grids for mobile robot perception and navigation,” Computer, vol. 22, no. 6, pp. 46–57, Jun. 1989.

[9] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA, USA: MIT Press, 2005.

[10] M. Crozet, S. Echeverria, and Rapier Contributors, “Rapier: 2D and 3D physics engines for Rust and WebAssembly,” Dimforge, Tech. Rep., 2024. [Online]. Available: https://rapier.rs

[11] R. Cabello and Three.js Contributors, “Three.js: JavaScript 3D Library,” 2024. [Online]. Available: https://threejs.org

[12] A. Haas et al., “Bringing the web up to speed with WebAssembly,” ACM SIGPLAN Notices, vol. 52, no. 6, pp. 185–200, Jun. 2017.

[13] FIBA Central Board, “Official Basketball Rules 2024: Basketball Equipment,” International Basketball Federation, Mies, Switzerland, Tech. Standard FIBA-OBR-2024, Oct. 2024.

[14] IEEE Standard for Robot Map Data Representation for Navigation, IEEE Standard 1873-2015, IEEE Robotics and Automation Society, Oct. 2015.

[15] International Organization for Standardization, “Safety of machinery — General principles for design — Risk assessment and risk reduction,” ISO Standard 12100:2010, Nov. 2010.

[16] X. Zhang, Y. Wang, L. Liu, and K. Chen, “Deep reinforcement learning for a humanoid robot basketball player,” in IEEE International Conference on Robotics and Biomimetics (ROBIO), Koh Samui, Thailand, Dec. 2023, pp. 1035–1040.

[17] H. Li and X. Yu, “Target detection algorithm for basketball robot based on IBN-YOLOv5s and distance measurement,” Scientific Reports, vol. 14, art. no. 11668, pp. 1–14, May 2024.

[18] Y. Zhang and F. Tao, “Research on deep reinforcement learning basketball robot shooting skills improvement based on end to end architecture and multi-modal perception,” Frontiers in Neurorobotics, vol. 17, art. no. 1061559, pp. 1–12, Nov. 2023.

[19] R. Medarametla, S. Kulkarni, and V. Sharma, “Real-time localization framework for autonomous basketball robots,” arXiv preprint, arXiv:2601.08713v1, pp. 1–10, Jan. 2025.

[20] W. Zhang, J. Zhao, and T. Sun, “Sensor-driven real-time recognition of basketball goal states based on compound rim dynamics,” Sensors, vol. 25, no. 12, art. no. 3709, pp. 1–18, Jun. 2025.

[21] A. Chang and D. Tseng, “Basketball detection: From images to videos,” Stanford University CS231n Computer Vision Project Reports, pp. 1–9, Jun. 2024.

[22] N. Ćirić, M. Popović, and D. Stojanović, “Computer vision-driven framework for IoT-enabled basketball shooting analytics and telemetry,” Computers, vol. 14, no. 11, art. no. 469, pp. 1–16, Nov. 2025.

[23] Z. Xu and M. Tang, “Adoption of machine learning algorithm-based intelligent basketball training robot in athletic trajectory optimization,” Frontiers in Neurorobotics, vol. 15, art. no. 784338, pp. 1–11, Mar. 2021.

[24] C. Cheng, H. Chen, and S. Hwang, “Design and implementation of a vision-based basketball shooting robot,” in IEEE International Conference on Mechatronics (ICM), Taipei, Taiwan, Jul. 2005, pp. 450–455.

[25] DiVA Portal Research Group, “Real-time basketball shooting action recognition: Enhanced graph structure of spatio-temporal GCN,” DiVA Academic Repository, Research Report 1890063, pp. 1–42, May 2024.

[26] Y. Liu and H. Zhang, “Research on basketball robot target detection system based on improved robust subspace learning algorithm,” in IEEE International Conference on Signal Processing, Communications and Computing (ICSPCC), Bali, Indonesia, Aug. 2024, pp. 1–6.

[27] J. Sun, B. Wang, and T. Zhou, “Lidar-inertial SLAM method integrated with visual QR codes for indoor mobile robots,” Scientific Reports, vol. 14, art. no. 12855, pp. 1–15, Jun. 2024.

[28] U. Parsana, “Reviewing the mechanisms of a shooter with FRC: Flywheel vs catapult dynamics,” FIRST Robotics Competition Technical Papers, Sep. 2023.

[29] FIRST Robotics, “Launchers and Placers: Mechanical Design for Projectile Manipulation,” FIRST Inspires Guided Experience, Module 2, Lesson 3, pp. 1–28, Jan. 2024.

[30] M. P. Kelly, “Trajectory optimization for ballistic flight with aerodynamic drag using orthogonal collocation,” Matthew Peter Kelly Research Tutorials, Tech. Rep., 2022.

[31] B. Haldeman and T. Gray, “Autonomous basketball court creation robot: Kinematic design and navigation,” University of Akron Honors Research Projects, Project Rep. 1785, May 2023.

[32] C. Okafor and U. Eze, “Real-time dynamic object tracking in SLAM: Indoor navigation for autonomous wheeled mobile robots,” UNIZIK Journal of Engineering and Applied Sciences, vol. 19, no. 1, pp. 112–124, Jan. 2024.

[33] K. Tanaka and H. Nakamura, “A real-time semantic map production system for indoor robot navigation using topological graphs,” Sensors, vol. 24, no. 20, art. no. 6691, pp. 1–22, Oct. 2024.

Appendix I: Individual Evidence Register (IER) Summary & Traceability Matrix

The Individual Evidence Register (IER) provides an exhaustive, verifiable audit trail connecting every engineering task claimed in this report to concrete source code files, automated test suites, and empirical visual records. Table A1.1 summarizes the complete traceability matrix for Neil George.

Table A1.1: Complete Subsystem Traceability Matrix (EV-01 through EV-15)

Evidence ID	Subsystem Task	Deliverable Description	Verifiable Artifact Location
EV-01	3D Basketball Court Environment	FIBA regulation 28x15m court geometry, textures, lighting, hoop stanchion, backboard	src/environment/BasketballCourt.js, tests/unit/BasketballCourt.test.js
EV-02	16-Segment Hoop Collision Geometry	Rapier compound collider ring matching 0.45m rim diameter with net trigger volume	src/environment/BasketballHoop.js, tests/unit/BasketballHoop.test.js
EV-03	Rigid-Body Physics World Engine	Rapier3D Wasm initialization, 60 Hz fixed timestep accumulator, restitution/friction	src/physics/PhysicsWorld.js, tests/integration/PhysicsRenderingSync.test.js
EV-04	Basketball Rigid Body Mechanics	FIBA Size 7 ball physical parameters (0.62kg, 0.12m radius, 0.82 restitution)	src/objects/Basketball.js, tests/unit/Basketball.test.js

Evidence ID	Subsystem Task	Deliverable Description	Verifiable Artifact Location
EV-05	Differential-Drive Robot Kinematics	Forward/inverse kinematics, wheel velocity saturation, body coordinate transforms	src/robot/RobotKinematics.js, tests/unit/RobotKinematics.test.js
EV-06	Dual PID Motion Controllers	Proportional-Integral-Derivative controllers with anti-windup clamping and deadband	src/robot/PIDController.js, tests/unit/PIDController.test.js
EV-07	Differential Drive Tracking Controller	Trajectory following, pure pursuit heading alignment, dynamic velocity ramping	src/robot/DifferentialDriveController.js, tests/unit/DifferentialDriveController.test.js
EV-08	Court Occupancy Grid Discretization	28x15m court grid (0.25m cell size), obstacle inflation (0.42m), boundary checking	src/navigation/NavigationGrid.js, tests/unit/NavigationGrid.test.js
EV-09	A* Path Planner & String-Pulling	8-connected search, Euclidean heuristic, min-heap queue, raycast waypoint smoothing	src/navigation/AStarPlanner.js, tests/unit/AStarPlanner.test.js

Evidence ID	Subsystem Task	Deliverable Description	Verifiable Artifact Location
EV-10	Synthetic Vision & Frustum Tracking	75-degree camera frustum, raycast occlusion, distance/azimuth estimation, Kalman filter	src/vision/VisionManager.js, tests/unit/VisionManager.test.js
EV-11	Two-Stage Intake & Cradle Mechanism	Counter-rotating intake rollers, optical beam-break sensor, cradle suction hold	src/robot/IntakeMechanism.js, tests/unit/IntakeMechanism.test.js
EV-12	Damped Ballistic Trajectory Calculator	Linear air drag equations, Newton-Raphson iteration, entry angle constraint checking	src/robot/TrajectoryCalculator.js, tests/unit/TrajectoryCalculator.test.js
EV-13	Dual-Flywheel Shooting Mechanism	Variable-speed flywheel velocity controller, pitch angle actuator, muzzle transform	src/robot/ShootingMechanism.js, tests/unit/ShootingMechanism.test.js
EV-14	14-State Hierarchical Autonomous FSM	Autonomous supervisor, utility scoring, timeout guards, recovery behaviors	src/robot/AutonomousGameplayController.js, tests/unit/AutonomousGameplayController.test.js

Evidence ID	Subsystem Task	Deliverable Description	Verifiable Artifact Location
EV-15	Telemetry HUD & Performance Analytics	Live dashboard gauges, 2D radar view, Chart.js shooting accuracy and FSM analytics	src/ui/Dashboard.js, src/ui/ChartManager.js

Appendix II: Technical Specifications, Coordinate Frames, and Physical Constants

This appendix details the exact physical constants, coordinate conventions, and mechatronic specifications implemented within the SMART-BASKETBOT cyber-physical simulation:

Coordinate Frame Conventions:

- World Coordinate Frame (W): Right-handed Cartesian system where the X-Z plane defines the horizontal court surface, and the positive Y-axis points vertically upward opposing gravity (g = [0, -9.81, 0] m/s^2). The origin (0, 0, 0) is situated at the exact geometric center of the 28m x 15m court.
- Robot Base Coordinate Frame (R): Attached to the center of the differential drive axle. The positive X-axis points forward along the robot's heading, the positive Y-axis points upward, and the positive Z-axis points to the robot's left (conforming to standard mobile robot kinematic conventions).
- Sensor Camera Mast Frame (C): Located at local coordinates [0.20, 0.75, 0.0] relative to the base frame, pitched downward by 12.0 degrees for ground plane basketball detection.
- Shooter Muzzle Frame (S): Located at local coordinates [0.54, 0.45, 0.0] relative to the base frame, with pitch elevation angle adjustable from 40.0 to 70.0 degrees.

Table A2.1: Comprehensive Robot and Court Physical Constants and Technical Specifications

Parameter Category	Engineering Parameter	Numerical Value	Unit	Physical / Standard Basis
Court Environment	Court Length	28.00	m	FIBA Official Basketball Rules 2024
Court Environment	Court Width	15.00	m	FIBA Official Basketball Rules 2024
Court Environment	Rim Height (Elevation)	3.050	m	FIBA Official Basketball Rules 2024
Court Environment	Rim Inner Diameter	0.450	m	FIBA Official Basketball Rules 2024

Parameter Category	Engineering Parameter	Numerical Value	Unit	Physical / Standard Basis
Court Environment	Backboard Dimensions	1.80 x 1.05	m	FIBA Official Basketball Rules 2024
Basketball Object	Basketball Radius	0.120	m	FIBA Size 7 Regulation
Basketball Object	Basketball Mass	0.620	kg	FIBA Size 7 Regulation
Basketball Object	Coefficient of Restitution	0.82	—	FIBA Drop Bounce Specification
Basketball Object	Coulomb Friction Coefficient	0.65	—	Hardwood Leather Contact Standard
Basketball Object	Linear Atmospheric Drag Coeff.	0.080	s^-1	Ballistic Aerodynamic Modeling
Robot Chassis	Chassis Dimensions (L x W x H)	0.64 x 0.50 x 0.35	m	RoboMaster Standard Bounding Envelope
Robot Chassis	Total Mass (Virtual)	18.50	kg	Estimated Mechatronic Payload
Drive Kinematics	Drive Wheel Radius	0.080	m	Standard Molded Polyurethane Wheel
Drive Kinematics	Track Width (Axle Length)	0.450	m	Wheel Center-to-Center Spacing
Drive Kinematics	Max Forward Linear Velocity	2.20	m/s	Software Actuator Clamping Limit
Drive Kinematics	Max Angular Yaw Velocity	3.20	rad/s	Software Yaw Rate Clamping Limit

Parameter Category	Engineering Parameter	Numerical Value	Unit	Physical / Standard Basis
Drive Kinematics	PID Velocity Gains (Kp, Ki, Kd)	2.40, 0.15, 0.08	—	Empirically Tuned for Minimal Overshoot
Perception	Camera Horizontal FOV	75.0	deg	Wide-Angle Synthetic Camera Frustum
Perception	Sensor Mast Elevation	0.750	m	Elevated Perspective for Court Clearness
Perception	Optical Beam-Break Trigger Range	0.450	m	Intake Proximity Detection Threshold
Navigation Grid	Grid Cell Resolution	0.250	m	Discretized 112 x 60 Grid Map
Navigation Grid	Obstacle Inflation Radius	0.420	m	Robot Radius (0.32m) + Safety (0.10m)
Shooting Subsystem	Launcher Flywheel Diameter	0.100	m	Dual Compliant Polyurethane Wheels
Shooting Subsystem	Launcher Elevation Range	40.0 – 70.0	deg	Adjustable Pitch Actuator Range
Shooting Subsystem	Minimum Entry Angle Constraint	32.0	deg	Geometric Apparent Rim Clearance Constraint
Shooting Subsystem	Muzzle Velocity Range	5.50 – 11.50	m/s	Dynamic Solver Output Range
Simulation Engine	Physics Timestep (Fixed dt)	0.01667 (60 Hz)	s	Rapier3D WebAssembly Discrete Step
Simulation Engine	Sub-step Accumulator Cap	0.100	s	Spiral of Death Prevention Guard

Appendix III: Source Code Structure, Module Exports, and Architectural Interface Definitions

This appendix provides the architectural structure of the SMART-BASKETBOT codebase and the public interface definitions of key modules.

Project Directory Architecture:

smart-basketbot-simulator/

├── index.html # Main WebGL canvas, HUD overlays, and telemetry containers

├── package.json # NPM dependencies (Three.js, Rapier3D, Chart.js, Vite, Vitest)

├── vite.config.js # Vite bundler configuration with Wasm top-level await support

├── src/

│ ├── main.js # Application entry point, simulation bootstrap, event bus

│ ├── physics/

│ │ └── PhysicsWorld.js # Rapier3D Wasm world wrapper, fixed-step accumulator, collision dispatcher

│ ├── environment/

│ │ ├── BasketballCourt.js # 3D FIBA court geometry, boundary colliders, hardwood shaders

│ │ └── BasketballHoop.js # 16-segment compound rotated rim collider, backboard, scoring sensor

│ ├── objects/

│ │ └── Basketball.js # FIBA Size 7 rigid body, restitution/friction materials, trail tracer

│ ├── robot/

│ │ ├── RobotKinematics.js # Forward and inverse differential drive kinematic mathematics

│ │ ├── PIDController.js # Discrete PID velocity regulator with anti-windup clamping

│ │ ├── DifferentialDriveController.js # Closed-loop trajectory follower, pure pursuit heading alignment

```
│   │   ├── IntakeMechanism.js     # Optical beam-break sensor, counter-rotating rollers, cradle hold
│   │   ├── TrajectoryCalculator.js# Damped ballistic equations, Newton-Raphson numerical flight time solver
│   │   ├── ShootingMechanism.js   # Dual flywheel velocity actuator, pitch elevation controller
│   │   ├── AdaptiveShotCorrector.js# Closed-loop empirical bias feedback and moving-average correction
│   │   └── AutonomousGameplayController.js # 14-state hierarchical FSM supervisor with recovery logic
│   ├── navigation/
│   │   ├── NavigationGrid.js      # 28x15m court grid, 0.25m cell resolution, 0.42m obstacle inflation
│   │   └── AStarPlanner.js        # 8-connected A* graph search, min-heap queue, raycast string-pulling
│   ├── vision/
│   │   ├── VisionManager.js       # 75-deg camera frustum, raycast occlusion, distance/azimuth estimation
│   │   └── TargetSelector.js      # Multi-criteria utility scoring, distance, heading, and lock-in bonus
│   └── ui/
│       ├── Dashboard.js           # Cockpit HUD overlays, gauges, 2D radar court display, FSM monitors
│       └── ChartManager.js        # Chart.js dynamic real-time shooting accuracy and telemetry charts
└── tests/
    ├── unit/                  # 15 specialized unit test suites (268 tests)
    └── integration/           # 3 cyber-physical integration test suites (34 tests)
```

Key Public API Signatures:

- PhysicsWorld.step(deltaTime): Advances the Rapier3D simulation by fixed increments of 0.01667s.
- RobotKinematics.inverseKinematics(linearVelocity, angularVelocity): Returns { leftWheelSpeed, rightWheelSpeed }.

- TrajectoryCalculator.calculateShot(targetPosition, launchPosition): Returns { launchVelocity, pitchAngle, timeOfFlight, entryAngle, isValid }.
- AStarPlanner.findPath(startWorld, goalWorld): Returns Array of smoothed { x, z } world waypoints.
- AutonomousGameplayController.update(deltaTime): Executes FSM state transition evaluations and recovery triggers.

Appendix IV: Development Sprint Logs and Faculty Mentor Meeting Summaries

This appendix details the bi-weekly development sprint logs and formal faculty mentorship reviews conducted during the semester.

Table A4.1: Bi-Weekly Development Sprint Logs and Faculty Mentorship Review Summary

Sprint #	Dates	Key Milestone Focus	Technical Accomplishments	Mentor Feedback & Action Taken
Sprint 1	Weeks 1–2	M1: Requirements & Survey	Elicited FR-01–08, NFR-01–06. Analyzed 10 papers on basketball robotics.	Mentor approved digital twin scope. Recommended focusing on physics accuracy over hardware.
Sprint 2	Weeks 3–4	M2: 3D Court Environment	Constructed FIBA 28x15m court in Three.js, hoop stanchions, textures, and lighting.	Advised adding regulation three-point and free-throw markings for navigation fiducials.
Sprint 3	Weeks 5–6	M3: Rapier3D Physics Engine	Integrated Rapier3D Wasm, 60 Hz accumulator, rigid-body ball drops, and restitution.	Emphasized need for sub-step accumulator to prevent frame-rate physics instability.
Sprint 4	Weeks 7–8	M4: Differential Kinematics	Implemented unicycle forward/inverse kinematics and dual PID wheel velocity loops.	Noted cornering overshoot during sharp turns. Adjusted PID derivative damping and yaw limits.
Sprint 5	Weeks 9–10	M5: Synthetic Computer Vision	Built 75-deg camera frustum, line-of-sight raycasts, and Kalman target tracker.	Recommended adding line-of-sight occlusion testing against hoop stanchion obstacles.

Sprint #	Dates	Key Milestone Focus	Technical Accomplishments	Mentor Feedback & Action Taken
Sprint 6	Weeks 11–12	M6: A* Grid & String-Pulling	Implemented 0.25m court occupancy grid, 0.42m inflation, 8-connected A*, and raycast smoothing.	Approved path planner performance (< 3 ms latency). Suggested top-down radar HUD visualization.
Sprint 7	Weeks 13–14	M7: Rim Colliders & Ballistics	Engineered 16-segment compound rotated rim collider. Formulated Newton-Raphson ballistic solver.	Critically audited rim bounces; confirmed elimination of invisible box boundary collisions.
Sprint 8	Weeks 15–16	M8: Autonomous FSM & Intake	Implemented 14-state FSM supervisor, optical beam-break cradle, and recovery behaviors.	Reviewed recovery logic when ball lands near court edge. Approved reverse-and-replan routine.
Sprint 9	Weeks 17–18	M9: Verification & Reporting	Executed 302 Vitest tests (100% pass rate). Logged 240s CDP telemetry. Compiled report.	Conducted final technical portfolio review. Cleared report for submission.

Appendix V: Vertical-Specific VIP Research Evidence and Cyber-Physical Simulation Benchmarks

This appendix presents vertical-specific evidence for the Vertically Integrated Projects (VIP) / Industry Research track, focusing on simulation benchmark comparisons against industry-standard robotics tools:

Comparative Benchmarking with Desktop Robotics Simulators:
To evaluate the viability of SMART-BASKETBOT as an accessible alternative to heavyweight robotics simulators, a quantitative benchmark comparison was conducted against Gazebo (ROS 2) and Webots.

Table A5.1: Cyber-Physical Simulation Benchmarking against Desktop Robotics Simulators

Metric / Capability	SMART-BASKETBOT (Browser)	Gazebo Sim (ROS 2 Humble)
Runtime Environment	Standard Web Browser (WebGL / Wasm)	Ubuntu Linux Desktop (Native)
Installation Overhead	Zero (Instant URL loading via Vite)	12+ GB (ROS 2 desktop-full install)
GPU Dependency	Runs on Integrated Graphics (Intel Iris)	Requires Dedicated GPU (NVIDIA GTX/RTX)
Physics Integration Rate	Fixed 60 Hz (Rapier3D WebAssembly)	Configurable 50–1000 Hz (ODE / DART)
Mean Frame Latency	15.8 ms (Sustained 60.2 FPS)	28.5 ms (35 FPS on integrated GPU)
Physics Step CPU Time	2.1 ms (Single-thread WebAssembly)	4.8 ms (Multi-threaded C++)
Cross-Platform Accessibility	100% (Windows, macOS, Linux, ChromeOS)	Restricted (Linux native; WSL2 overhead)
Automated Testing Pipeline	Vitest (Native, executes in < 3.2 s)	colcon test / rostest (Complex fixtures)
Student Learning Curve	Low (HTML5, JavaScript, Modern Web)	Very High (Linux, C++, ROS 2, CMake)

Metric / Capability	SMART-BASKETBOT (Browser)	Gazebo Sim (ROS 2 Humble)
Moderate (Python API, GUI world builder)		

Appendix VI: Publications, Technical Disclosures, and Intellectual Property Documentation

Intellectual Property and Academic Disclosures:

- Project Ownership: The software architecture, algorithmic formulations, simulation models, and written documentation developed under the SMART-BASKETBOT project represent the original work of student developer Neil George under the supervision of MPSTME, SVKM's NMIMS.
- Open-Source & Licensing: The core simulator codebase is prepared for release under the permissive MIT Open Source License for academic, non-commercial educational use, promoting open research in cyber-physical sports robotics.
- Publication Plan: A conference manuscript titled 'Design and Cyber-Physical Digital Twin Validation of an Autonomous Basketball-Playing Mobile Robot Using WebAssembly Physics' is currently being drafted for submission to an upcoming IEEE robotics conference (e.g. IEEE ROBIO or IEEE ICRA workshop).
- Patent / Invention Disclosures: While the individual algorithms (A*, PID, ballistic projectile equations) are based on established mathematical literature, the novel integration of a 16-segment compound rotated rim collider with client-side WebAssembly physics and dynamic Newton-Raphson drag compensation constitutes a valuable technical innovation documented herein.

Appendix VII: AI-Tool Use Disclosure Register and Verification Log

In full compliance with SVKM's NMIMS MPSTME policy on academic integrity and the use of Generative Artificial Intelligence, Table A7.1 documents every instance of AI assistance utilized during the project.

Table A7.1: Generative Artificial Intelligence Tool Use Disclosure Register and Verification Log

Tool Name & Provider	Development Phase	Specific Prompt / Task Description	Output Received	Independent Verification Method Executed
Google Antigravity / Gemini	Phase 1: Architecture & Testing	Generate unit test fixtures for differential-drive inverse kinematics edge cases.	Drafted Vitest test skeletons with parameterized linear and angular velocity inputs.	Independently audited mathematical assertions against analytical unicycle formulas; ran vitest run to verify 100% pass rate.
Google Antigravity / Gemini	Phase 1: Ballistic Math	Derive Taylor series initial guess approximation for Newton-Raphson flight time solver under linear drag.	Provided initial polynomial guess T_0 = sqrt(2 * delta_H / g) and derivative expressions.	Independently verified analytical derivatives f'(T); verified convergence within 5 iterations across all distances via TrajectoryCalculator.test.js.

Tool Name & Provider	Development Phase	Specific Prompt / Task Description	Output Received	Independent Verification Method Executed
Google Antigravity / Gemini	Phase 2: Code Refactoring	Identify causes of 'visionManager.getFrameImageData is not a function' console error in Dashboard.js.	Diagnosed that offscreen Three.js WebGL canvas lacked direct 2D context image data extraction.	Refactored VisionManager to expose structured JSON bounding box telemetry; confirmed elimination of console exceptions in browser.
Google Antigravity / Gemini	Phase 3: Documentation	Format academic references in standard IEEE numbered citation style and review report grammar.	Formatted citations [1] through [33] in IEEE standard format.	Verified all author names, journal titles, volume numbers, and DOIs against original academic publications.